



National University of Computer and Emerging Sciences, Lahore



Resolve AI

Muhammad Ali Shahzad 23L-0719 BS(CS)

Muhammad Ahmed 23L-0566 BS(CS)

Zohaib Tariq 23L-0979 BS(CS)

Supervisor: Junaid Hussain Muzammil

Department of Computer Science

Final Year Project – Deliverable II

October 5, 2026

Anti-Plagiarism Declaration

This is to declare that the above publication was produced under the:

Title: Resolve AI

is the sole contribution of the author(s), and no part hereof has been reproduced as it is the basis (cut and paste) that can be considered Plagiarism. All referenced parts have been used to argue the idea and cited properly. I/We will be responsible and liable for any consequence if a violation of this declaration is determined.

Date: .....

Name: Muhammad Ali Shahzad (23L-0719)

Signature: .....

Name: Muhammad Ahmed (23L-0566)

Signature: .....

Name: Zohaib Tariq (23L-0979)

Signature: .....

Author’s Declaration

This states Authors’ declaration that the work presented in the report is their own, and has not been submitted/presented previously to any other institution or organization.

Abstract

Pakistani e-commerce sellers often manage returns and complaints through spreadsheets, calls, and WhatsApp messages. Resolve AI proposes a Shopify-integrated platform combining return-risk prediction with English and Roman Urdu complaint handling. Gradient-boosted trees will estimate return/refund risk using real Pakistani order outcomes, while a multilingual transformer will classify complaint category and urgency. Sentiment analysis and retrieval-augmented generation will support responses grounded in seller policies. An orchestration layer will route cases to automatic resolution, escalation, or human agents. A dashboard and WhatsApp Cloud API integration will support monitoring and communication. The expected outcome is a demonstrable system that assists sellers with post-purchase decisions while retaining human oversight.

Executive Summary

Small and mid-sized Pakistani e-commerce sellers face recurring difficulties in managing returns, refund requests, and customer complaints. Order records and customer conversations may be spread across spreadsheets, phone calls, and WhatsApp threads, making it difficult to identify cases requiring attention and provide consistent responses. Complaints written in Roman Urdu or a mixture of Urdu and English further complicate automated interpretation. Resolve AI addresses these problems through a proposed post-purchase intelligence platform that connects order information with bilingual complaint analysis and policy-grounded assistance.

The platform will ingest order, return, and refund events from Shopify and normalize them for downstream processing. A return-risk model based on XGBoost or LightGBM will be developed using Pakistan's Largest E-Commerce Dataset, which contains historical transactions from a Pakistani online store. The prediction target will be defined after examining the available status labels. Risk scores will flag orders for review rather than determine fraud or justify automatic rejection. The dataset's historical period and single-store origin will be considered when evaluating how well the model may transfer to other sellers.

Incoming text complaints will be assigned a category and urgency level using a fine-tuned multilingual transformer, such as XLM-RoBERTa or mBERT. A pretrained sentiment model will provide an additional signal about the customer's tone. The resolution-suggestion module will use sentence embeddings, Qdrant, and a large language model to retrieve relevant passages from the seller's return and refund policy and draft an appropriate response. The orchestration layer will combine complaint analysis, risk information, and case history to route cases to automatic resolution, escalation, or a human agent. The permitted automatic actions and escalation conditions will be defined in the requirements and design.

An analytics dashboard will summarize return trends, risk tiers, complaint categories and urgency, and courier performance where suitable data are available. WhatsApp Cloud API integration will demonstrate notifications and complaint intake.

Table of Contents

List of Figures **ix**

List of Tables **xi**

1 Introduction **1**

1.1 Purpose of this Document 1

1.2 Intended Audience 1

1.3 Definitions, Acronyms, and Abbreviations 2

1.4 Conclusion 3

2 Project Vision **4**

2.1 Problem Domain Overview 4

2.2 Problem Statement 4

2.3 Problem Elaboration 5

2.3.1 Fragmented Order and Complaint Information 5

2.3.2 Limited Visibility of Return and Refund Risk 5

2.3.3 Bilingual and Informal Complaint Language 5

2.3.4 Inconsistent Prioritization and Case Assignment 5

2.3.5 Responses That Do Not Reflect Seller Policies 5

2.3.6 Limited Combined Operational Visibility 6

2.4 Goals and Objectives 6

2.5 Project Scope 7

2.6 Sustainable Development Goal (SDG) 7

2.7 Constraints 8

2.8 Business Opportunity 9

2.9 Stakeholders Description and User Characteristics 9

2.9.1 Stakeholders Summary 9

2.9.2 Key High-Level Goals and Problems of Stakeholders 10

2.10 Conclusion 10

3 Related Applications **11**

3.1 Definitions, Acronyms, and Abbreviations . . . 11

3.2 Relevant Techniques . . . 11

3.2.1 XGBoost . . . 11

3.2.2 LightGBM . . . 12

3.2.3 Multilingual BERT Transfer . . . 12

3.2.4 XLM RoBERTa . . . 12

3.2.5 Sentence BERT . . . 12

3.2.6 Retrieval Augmented Generation . . . 13

3.2.7 XLM T . . . 13

3.2.8 Pakistani Transaction Data and Vector Retrieval . . . 13

3.3 Existing Applications . . . 13

3.3.1 Loop Returns . . . 14

3.3.2 Zendesk AI Agents . . . 14

3.3.3 Gorgias AI Agent . . . 14

3.3.4 Intercom Fin . . . 14

3.3.5 PKAILabs . . . 15

3.3.6 ShuttlePro . . . 15

3.4 Summary Tables . . . 15

3.5 Conclusion . . . 17

4 Software Requirement Specifications **18**

4.1 List of Features . . . 18

4.2 Functional Requirements . . . 18

4.2.1 Seller and Store Owner . . . 18

4.2.2 Store Manager and Support Agent . . . 19

4.2.3 Customer . . . 19

4.2.4 External Services and Automated Processing . . . 20

4.3 Quality Attributes . . . 20

4.4 Non-Functional Requirements . . . 21

4.5 Assumptions . . . 22

4.6 Use Cases . . . 23

4.7 Hardware and Software Requirements . . . 30

4.7.1 Hardware Requirements . . . 30

4.7.2 Software Requirements . . . 31

4.8 Graphical User Interface 32
4.8.1 Account Access 33
4.8.2 Store, Messaging, and Team Settings 35
4.8.3 Analytics Dashboard 38
4.8.4 Orders and Return/Refund Risk 40
4.8.5 Complaint Queue and Case Detail 42
4.8.6 Seller Policy Library 43
4.9 Database Design 43
4.9.1 ER Diagram 44
4.9.2 Data Dictionary 44
4.10 Risk Analysis 51
4.11 Conclusion 53

5 Proposed Approach and Methodology **54**
5.1 Development Approach 54
5.2 Data Preparation and Model Development 54
5.3 Policy-Grounded Complaint Resolution 55
5.4 Integration and Evaluation 55
5.5 Conclusion 55

6 High-Level and Low-Level Design **56**
6.1 System Overview 56
6.2 Design Considerations 56
6.2.1 Assumptions and Dependencies 56
6.2.2 General Constraints 56
6.2.3 Goals and Guidelines 56
6.2.4 Development Methods 57
6.3 System Architecture 57
6.4 Architectural Strategies 58
6.5 Domain Model/Class Diagram 58
6.6 Sequence Diagram 59
6.7 Policies and Tactics 60
6.8 Conclusion 60

Bibliography **61**

List of Figures

2.1 **SDG 9: Industry, Innovation and Infrastructure, represented by an orange icon with three interconnected white blocks** 8

4.1 **Sign-in screen for sellers and staff showing email and password fields (UC-02)** . . . 33
4.2 **Account registration screen showing name, email, and password fields (UC-01)** . . . 34
4.3 **Store and messaging settings showing Shopify and WhatsApp connection configuration (UC-03 and UC-04)** 35
4.4 **Team-management screen showing staff memberships, roles, and access controls** . . 36
4.5 **Complaint-routing settings showing automatic-reply, urgency, and confidence controls** 37
4.6 **Analytics overview showing order counts, risk distribution, and priority conversations (UC-12)** 38
4.7 **Additional dashboard panels showing flagged orders, complaint categories, order outcomes, and courier coverage (UC-12)** 39
4.8 **Order-risk list showing customer records, scores, tiers, and observed outcomes (UC-05)** 40
4.9 **Selected-order detail panel showing risk score, customer history, and prediction metadata (UC-05)** 41
4.10 **Complaint workspace showing a Roman Urdu conversation, analysis, policy evidence, reply editor, and case actions (UC-07 to UC-10)** 42
4.11 **Seller policy library showing policy content, version history, and activation controls (UC-06)** 43
4.12 **Entity Relationship Diagram of Resolve AI** 44

6.1 **Proposed Resolve AI architecture showing staff access, external integrations, persistent storage, and background model processing** 57
6.2 **Resolve AI service class diagram showing application responsibilities, entity usage, and service dependencies** 59

6.3 Complaint-handling sequence with policy-grounded response generation, conditional routing, staff review, and delivery recording 60

List of Tables

2.1 Resolve AI stakeholders and their responsibilities . . . 9

3.1 Related applications and their relevance to Resolve AI . . . 16
3.2 Research studies supporting Resolve AI development . . . 16

4.1 Non-functional requirements and verification conditions . . . 21
4.2 UC-01 Create seller account . . . 23
4.3 UC-02 Sign in and sign out . . . 24
4.4 UC-03 Connect Shopify store . . . 24
4.5 UC-04 Synchronize order outcomes . . . 25
4.6 UC-05 Review return/refund-risk score . . . 25
4.7 UC-06 Maintain seller policies . . . 26
4.8 UC-07 Submit a WhatsApp complaint . . . 26
4.9 UC-08 Review complaint analysis . . . 27
4.10 UC-09 Review and send response . . . 28
4.11 UC-10 Route and resolve a complaint . . . 28
4.12 UC-11 Receive customer notifications . . . 29
4.13 UC-12 Review analytics . . . 30
4.14 Proposed software components . . . 31
4.15 Store data dictionary . . . 45
4.16 User data dictionary . . . 45
4.17 Membership data dictionary . . . 45
4.18 Customer data dictionary . . . 46
4.19 Order data dictionary . . . 46
4.20 SourceEvent data dictionary . . . 47
4.21 RiskScore data dictionary . . . 47
4.22 ComplaintCase data dictionary . . . 47
4.23 Message data dictionary . . . 48
4.24 ComplaintAnalysis data dictionary . . . 49

4.25 PolicyDocument data dictionary 49
4.26 PolicyChunk data dictionary 49
4.27 ResponseSuggestion data dictionary 50
4.28 Notification data dictionary 50
4.29 AuditEntry data dictionary 51
4.30 Technical and business risks and mitigations 51

Chapter 1 Introduction

Resolve AI is a proposed post-purchase intelligence platform for small and mid-sized Pakistani e-commerce sellers. It brings together return-risk prediction and bilingual complaint handling in a single workflow connected to Shopify. The project addresses situations in which sellers must review order outcomes, interpret customer messages, and consult return or refund policies across separate records and conversations.

Customer complaints may be written in English, Roman Urdu, or a mixture of Urdu and English. Differences in spelling and informal expressions can make these messages difficult to categorize consistently. For example, a customer requesting an exchange may describe the same issue using different vocabulary or switch languages within a sentence. Resolve AI will analyse complaint text to identify its category, urgency, and sentiment, then retrieve relevant seller-policy passages to support a resolution suggestion.

The platform will also estimate return/refund risk from order information using a gradient-boosted tree model developed with historical Pakistani transaction data. The score will support order review and will not be treated as proof of fraud. Shopify integration will provide order, return, and refund events, while WhatsApp Cloud API integration will support complaint intake and customer notifications. An orchestration layer will route cases to automatic resolution, escalation, or a human agent, and an analytics dashboard will summarize the resulting operational information.

1.1 Purpose of this Document

This document provides a technical and general overview of our Final Year Project, Resolve AI, a web-based platform for predicting return/refund risk and managing customer complaints for small and mid-sized Pakistani e-commerce businesses. The project aims to develop a working Shopify-integrated application that predicts return/refund risk, classifies complaints in English and Roman Urdu, and suggests responses based on the seller's return and refund policies. This document explains the project's problem domain, objectives, scope, related applications, software requirements, and high-level and low-level design. It provides an understanding of how the platform will assist sellers with complaint prioritization, case routing, customer communication, and monitoring through an analytics dashboard.

1.2 Intended Audience

The intended audience for this document includes the project advisor and academic evaluation committee who will review the objectives, requirements, and design of Resolve AI. The project team will also use it as a reference for developing and testing the application. In addition, e-commerce sellers, store

managers, and customer-support agents can review the report to understand how the platform will support order-risk review, bilingual complaint handling, policy-based resolution suggestions, and customer notifications. Customers are indirect beneficiaries of the system, receiving responses and updates about their orders and complaints through WhatsApp.

1.3 Definitions, Acronyms, and Abbreviations

The following terms establish the terminology used throughout the report.

FYP Final Year Project.

AI Artificial Intelligence.

ML Machine Learning.

NLP Natural Language Processing, the processing and analysis of human language by computer systems.

API Application Programming Interface, an interface through which software systems exchange data or invoke functionality.

CRM Customer Relationship Management. CRM-agnostic refers here to a design that does not require a particular CRM product; Shopify remains the agreed commerce integration.

RAG Retrieval-Augmented Generation, a method that supplies retrieved reference content to a language model to support its response.

LLM Large Language Model, a model used in this project to draft responses using retrieved seller-policy content.

XGBoost Extreme Gradient Boosting, a gradient-boosted tree framework considered for return-risk modelling.

LightGBM Light Gradient Boosting Machine, an alternative gradient-boosted tree framework.

XLM-R XLM-RoBERTa, a multilingual pretrained transformer considered for complaint classification.

mBERT Multilingual Bidirectional Encoder Representations from Transformers, an alternative multilingual pretrained model.

Roman Urdu Urdu written using the Latin alphabet, often with variable or informal spelling.

Code switching The use of more than one language within the same message or conversation.

Return risk An estimated likelihood of an order ending in a return or refund, subject to the outcome

labels available for training. It does not establish fraudulent intent.

Complaint ticket A record of a customer issue, its associated messages, analysis, assignment, and handling status.

Sentiment analysis Analysis of the emotional tone expressed in complaint text, used as an additional handling signal.

Webhook An event-triggered notification sent by one service to another, such as a Shopify order-event notification.

Sentence embedding A numerical representation of text used to retrieve semantically related policy passages.

Qdrant The vector database proposed for storing and searching policy embeddings.

Orchestration Coordination of analysis and case-handling components to determine the next permitted action.

Escalation Routing a case for additional attention when routine handling is insufficient.

Human agent A seller's staff member who reviews or handles customer complaints, distinct from an automated AI component.

UI User Interface.

SDG Sustainable Development Goal.

1.4 Conclusion

This chapter introduced Resolve AI as a platform combining return-risk prediction with English and Roman Urdu complaint handling. It established the purpose of the report, identified its intended audience, and defined the terminology needed to understand the proposed system. The following chapters develop the project vision and translate it into requirements and design.

Chapter 1 introduces the project and the purpose of this document. Chapter 2 presents the problem domain, goals, scope, constraints, and stakeholders. Chapter 3 reviews related applications and research techniques and explains their relevance to Resolve AI. Chapter 4 specifies the software requirements, use cases, interfaces, data design, and project risks. Chapter 5 describes the proposed approach and methodology for development and evaluation. Chapter 6 presents the high-level and low-level design, including the architecture and component interactions.

Chapter 2 Project Vision

This chapter presents the vision of Resolve AI and explains the operational problems that the project aims to address. It defines the objectives and scope of the application, its relationship to sustainable development, the constraints on its development, and the needs of its stakeholders. The vision is to provide Pakistani e-commerce sellers with a working platform that brings order-risk review and bilingual complaint handling into a coordinated post-purchase workflow.

2.1 Problem Domain Overview

Post-purchase operations include responding to customer concerns, reviewing return and refund requests, communicating order updates, and monitoring the outcomes of completed transactions. In the seller workflow addressed by this project, order information is available through an e-commerce store, while complaint conversations and follow-up activities may be handled separately. A support agent may need to locate an order, interpret a message, consult the seller's policy, and determine the next action before responding to a customer.

Resolve AI will connect to Shopify to receive order, return, and refund events and organize these records for analysis. It will estimate return/refund risk from order features and present scores for seller review. The platform will also receive text complaints through WhatsApp, classify their category and urgency, analyse sentiment, and retrieve relevant return or refund policy passages to support a response. The routing layer will determine whether a case is suitable for automatic handling or requires escalation or a human agent. Sellers will use the dashboard to review cases and monitor return and complaint trends.

The platform is intended to complement the connected store's order-management capabilities. Its focus is the combination of return-risk analysis, English and Roman Urdu complaint understanding, and policy-grounded assistance. Its internal case-handling workflow will not require a particular CRM product, although Shopify is the commerce integration included in the agreed scope.

2.2 Problem Statement

Small and mid-sized Pakistani e-commerce sellers need a coordinated way to review return/refund risk and handle customer complaints written in English and Roman Urdu. When order records, customer messages, and policy decisions are handled separately, sellers may struggle to prioritize cases and provide consistent responses. Resolve AI aims to address this problem by combining order-risk prediction, bilingual complaint analysis, policy-grounded resolution suggestions, and case routing in a Shopify-integrated platform.

2.3 Problem Elaboration

The overall problem consists of the following operational difficulties, each of which informs a component of the proposed system.

2.3.1 Fragmented Order and Complaint Information

A complaint cannot always be handled from its message alone. The agent may need the customer's order details, previous messages, and return or refund status. Reviewing these separately introduces repeated searches and creates opportunities for missed context. Resolve AI will organize available order and complaint information so that cases can be reviewed within a consistent workflow.

2.3.2 Limited Visibility of Return and Refund Risk

Historical order outcomes can reveal patterns associated with returns or refunds, but reviewing individual records manually does not provide a consistent estimate for new orders. The project will develop a model that presents an estimated risk score using features available at the time of prediction. The score will support review rather than establish customer misconduct. Its usefulness will depend on valid outcome labels and evaluation on data excluded from training.

2.3.3 Bilingual and Informal Complaint Language

Customer messages may combine English and Urdu vocabulary or use Roman Urdu with inconsistent spelling. These variations can make the same complaint appear different across messages. A multilingual classifier will be fine-tuned to assign category and urgency labels, with evaluation covering English and Roman Urdu or Urdu-English text.

2.3.4 Inconsistent Prioritization and Case Assignment

Handling complaints only in arrival order can delay cases that need prompt attention. A customer expressing frustration may also require a different response from a routine information request. Resolve AI will use urgency, sentiment, relevant risk information, and case history to support prioritization and routing. Human review will remain available when the analysis is uncertain or the required action exceeds permitted automatic handling.

2.3.5 Responses That Do Not Reflect Seller Policies

Generic replies may overlook the seller's return window, eligibility conditions, or refund procedure. Resolve AI will retrieve relevant passages from the seller's policy documents before drafting a resolution

suggestion. If the available policy does not support an answer, the case should be referred for review rather than answered through an unsupported commitment.

2.3.6 Limited Combined Operational Visibility

Order reports and separate complaint conversations do not directly provide a combined view of predicted risk, complaint categories, urgency, and case outcomes. The dashboard will bring these indicators together. Courier performance will be summarized only where the ingested records provide suitable courier and delivery information.

2.4 Goals and Objectives

The overall goal is to develop a working post-purchase intelligence platform that assists Pakistani e-commerce sellers with return-risk review and bilingual complaint management. The specific objectives are to:

- Implement a Shopify-integrated pipeline that receives and normalizes order, return, and refund events.
- Develop an XGBoost or LightGBM model to estimate return/refund risk using historical Pakistani order outcomes and assess its predictive performance.
- Fine-tune a multilingual transformer to classify complaint category and urgency for English and Roman Urdu or Urdu-English messages.
- Integrate a pretrained sentiment model as an additional signal for complaint handling.
- Build a resolution-suggestion module using sentence embeddings, Qdrant, and an LLM to retrieve seller-policy passages and draft grounded responses.
- Implement case routing to automatic resolution, escalation, or human agents according to defined handling conditions.
- Develop a dashboard showing return trends, risk tiers, complaint categories and urgency, and courier performance where supported by available data.
- Integrate WhatsApp Cloud API for complaint intake and order, return, and complaint notifications.

2.5 Project Scope

Resolve AI will be developed as a web-based platform connected to Shopify. The project scope includes the following:

- A Shopify-integrated data pipeline that receives order, return, and refund events through webhooks and normalizes them into records used by the application.
- A return-risk model using XGBoost or LightGBM, developed using Pakistan's Largest E-Commerce Dataset, with predictions presented as review signals.
- A bilingual complaint classifier that assigns category and urgency labels to English, Roman Urdu, and Urdu-English code-switched text using a fine-tuned multilingual transformer.
- A sentiment-analysis module using a pretrained model to analyse the customer's tone.
- A policy-grounded resolution-suggestion module that uses sentence embeddings, Qdrant, and an LLM to retrieve relevant return and refund policy content and draft a supported response.
- An orchestration layer that coordinates analysis and routes cases to automatic resolution, escalation, or a human agent. Automatic resolution refers to complaint responses within defined handling conditions. The system does not process returns, issue refunds, arrange exchanges, or manage return shipping. Exception handling will be specified in the software requirements and design chapters.
- An analytics dashboard presenting return and complaint trends, risk tiers, category and urgency distributions, and courier performance where suitable records are available.
- WhatsApp Cloud API integration supporting complaint intake and customer notifications.

The following activities are outside the project scope:

- Fraud or return-abuse classification.
- Voice-note or call transcription.
- Damaged-item photo verification.

2.6 Sustainable Development Goal (SDG)

Resolve AI is aligned with SDG 9, Industry, Innovation and Infrastructure, through the development of software that supports digital business operations. The project applies existing AI techniques to the practical needs of small and mid-sized e-commerce sellers, providing a means to organize complaint

handling and use order information in operational decisions. This is an intended contribution to business process innovation; any improvement in seller efficiency will need to be established through evaluation rather than assumed from the system's design.

Figure 2.1 identifies the Sustainable Development Goal associated with the project.



Figure 2.1: SDG 9: Industry, Innovation and Infrastructure, represented by an orange icon with three interconnected white blocks

2.7 Constraints

The project will be developed over two semesters by a three-member team. Development must therefore balance application integration, dataset preparation, model training, and documentation within the available time. Access to suitable computing resources will also affect transformer fine-tuning and the choice of model sizes.

Data availability is a major constraint. The Pakistani transaction dataset covers historical records and does not establish the behaviour of every current seller. Training labels and prediction-time features must be checked to avoid misleading targets or information leakage. Complaint classification also requires appropriately labelled English and Roman Urdu examples, while sentiment-model suitability must be assessed on the language used by the intended customers.

Shopify and WhatsApp integrations depend on account configuration, API access, network availability, and service limits. Model-generated responses depend on the completeness and currency of the seller's policies. Courier analytics depend on the fields available in the connected records. Customer and order information must be handled with access controls appropriate to the seller's staff roles.

2.8 Business Opportunity

Resolve AI offers a business opportunity as a helpful tool for small and mid-sized Pakistani e-commerce sellers seeking to reduce manual complaint handling and review return/refund-risk scores. The platform aims to reduce the effort required to read, categorize, prioritize, and respond to customer complaints, while analysing return/refund risk using order information and available customer purchase and return history. It will support personalized responses by considering the customer's complaint, relevant order details, previous interactions, and the seller's return and refund policies, reducing reliance on generic response templates. By bringing these capabilities together with English and Roman Urdu complaint handling, Resolve AI aims to help sellers manage customer issues more efficiently and provide more relevant and consistent support.

2.9 Stakeholders Description and User Characteristics

The main direct users are sellers, store managers, and customer-support agents. They need clear case information and understandable recommendations without having to operate the underlying models. Customers interact through messages and notifications, while the project team and advisor are responsible for development and academic review.

2.9.1 Stakeholders Summary

Table 2.1 summarizes the stakeholders and their relationship to Resolve AI.

Table 2.1: Resolve AI stakeholders and their responsibilities

Stakeholder	**Role and interest**
Seller or store owner	Reviews operational trends and risk information and provides the policies used for complaint assistance.
Store manager	Coordinates complaint handling, reviews escalations, and monitors case progress.
Customer-support agent	Reviews assigned cases, checks suggested responses, and handles complaints requiring human attention.
Customer	Submits complaints and receives responses and updates concerning an order or request.
Development team	Builds and tests the integrations, models, dashboard, and case-handling workflow.
Project advisor and evaluators	Review the objectives, scope, design, and evidence of functional and technical achievement.

2.9.2 Key High-Level Goals and Problems of Stakeholders

Sellers and store managers need visibility into return and complaint activity so that they can identify cases requiring attention. Their main problem is coordinating order information and customer conversations while maintaining consistent policy decisions.

Customer-support agents need the complaint, relevant order context, and applicable policy information together. They require useful category and urgency labels, editable suggestions, and clear escalation paths. Incorrect classifications or unsupported replies could increase their workload, so they must be able to review and correct the system's recommendations.

Customers need clear responses and updates appropriate to their issue. Their messages should be interpreted even when written informally or using Roman Urdu. A case requiring a human decision should reach an agent rather than remain blocked in an automatic workflow.

2.10 Conclusion

This chapter defined the vision of Resolve AI as a working platform for order-risk review and bilingual complaint handling. It connected the operational problems to eight deliverables, established the project boundaries and constraints, and described the needs of the stakeholders. The next chapter examines related applications and techniques to inform the requirements and design of the proposed platform.

Chapter 3 Related Applications

This chapter reviews applications and techniques relevant to the development of Resolve AI. It examines customer-support and returns-management platforms, then considers established methods for tabular prediction, multilingual classification, sentiment analysis, retrieval, and evaluation. The purpose is to inform implementation choices and identify what must be demonstrated by the proposed system, rather than claim a new learning algorithm or assume that competing products lack comparable features.

3.1 Definitions, Acronyms, and Abbreviations

The following terms supplement the definitions in Chapter 1.

Helpdesk Software that organizes customer conversations and support cases.

Self-service return portal An interface through which customers submit return requests.

Transfer learning Adaptation of representations learned from one dataset or task to another.

Fine-tuning Training a pretrained model on examples for a particular task.

Macro F1 An unweighted average of per-class F1 scores, useful when classes have unequal frequencies.

3.2 Relevant Techniques

This section reviews the techniques directly relevant to return/refund-risk scoring, bilingual complaint classification, sentiment analysis, and policy-grounded response suggestions. It also identifies the transaction dataset and vector database used by the proposed system.

3.2.1 XGBoost

Chen and Guestrin describe a scalable gradient-boosted tree system with sparsity-aware processing and efficient tree learning. [1]

It provides a practical approach to tabular prediction, but useful performance still depends on valid features, representative data, and parameter tuning.

Resolve AI will consider XGBoost for return-risk scoring and compare its results with a baseline. Customer-history features will use only information available before the scored order.

3.2.2 LightGBM

Ke et al. introduce gradient-based one-side sampling and exclusive feature bundling to improve the efficiency of gradient-boosted tree training. [2]

Efficient training makes the framework attractive for larger tabular datasets. Its reported gains do not establish performance on Pakistani order outcomes.

LightGBM is an alternative for the return-risk component. Selection between it and XGBoost will depend on validation performance and resource use.

3.2.3 Multilingual BERT Transfer

Pires et al. examine cross-lingual transfer in multilingual BERT, including transfer across languages and scripts. [3]

The results support multilingual representation sharing, but transfer varies with linguistic differences. Multilingual coverage alone does not guarantee informal code-switched accuracy.

mBERT will be considered as an alternative complaint-classification model. English and Roman Urdu results will be assessed separately.

3.2.4 XLM RoBERTa

Conneau et al. present XLM-R, a multilingual masked language model trained on data from one hundred languages. [4]

Its cross-lingual results make it a relevant starting point for bilingual tasks. Performance on Urdu benchmarks does not automatically establish performance on Latin-script Roman Urdu.

Resolve AI will consider XLM-R for complaint category and urgency classification, using labelled examples that reflect its intended language mix.

3.2.5 Sentence BERT

Reimers and Gurevych introduce sentence embeddings that can be compared efficiently for semantic similarity. [5]

Reusable embeddings support retrieval, but similarity alone does not establish policy applicability. Language coverage depends on the selected embedding model.

Resolve AI will use a suitable sentence-embedding model to match complaints with policy passages. Retrieval quality must be checked on relevant bilingual queries.

3.2.6 Retrieval Augmented Generation

Lewis et al. combine retrieval from external knowledge with language generation for knowledge-intensive tasks. [6]

Retrieval provides an external information source, but does not guarantee that every generated statement is supported by retrieved content.

Resolve AI will adapt this principle to seller policies. Suggestions should refer to applicable policy passages, with review when evidence is missing or conflicting.

3.2.7 XLM T

Barbieri et al. adapt multilingual language modelling to Twitter and provide multilingual sentiment resources. [7]

The work is relevant to informal messages. Its multilingual sentiment evaluation does not establish reliable Roman Urdu complaint sentiment.

It provides a candidate family for the pretrained sentiment module. Sentiment will supplement urgency and category, rather than determine the resolution alone.

3.2.8 Pakistani Transaction Data and Vector Retrieval

Pakistan's Largest E-Commerce Dataset, published by Usmani, is the transaction source identified in the proposal. [8] Its local origin makes it relevant to the return-risk component, but historical data from one seller does not establish generalization to every store. Data preparation must check the available customer identifiers and whether prior history can be constructed without using future outcomes. Missing customer history should be handled explicitly rather than fabricated.

Qdrant supports vector retrieval and metadata filtering. [9] Resolve AI will use it to search policy embeddings while retaining document and seller context. The database supplies retrieval infrastructure; response quality also depends on suitable embeddings, policy preparation, and the language model. Filtering by seller and applicable policy version is a design consideration for avoiding unrelated policy content.

3.3 Existing Applications

The following six platforms are compared with Resolve AI. The comparisons distinguish documented features from unverified performance. Resolve AI's return/refund component provides risk scores; it

does not process returns or issue refunds.

3.3.1 Loop Returns

Loop's Return Prediction Dashboard includes forecasts of return volumes and an order-level view showing the probability that an order will have an item returned. Its documented risk model uses product, customer-history, and order-context information. [10]

This directly overlaps with Resolve AI's risk-scoring feature. The documentation describes order-level prediction as a pilot for a limited enterprise group, with daily updates and historical-data requirements.

Resolve AI will develop return/refund-risk scoring using Pakistani transaction data and connect scores with bilingual complaint assistance. Loop's return-processing capabilities are not part of Resolve AI.

3.3.2 Zendesk AI Agents

Zendesk provides knowledge-based reply assistance and support workflows. Its AI-agent language documentation lists Urdu among supported languages. [11] [12]

The same documentation explicitly states that transliterated language input is unsupported and requires the native script. This is a documented limitation relevant to Urdu written in Latin characters.

Resolve AI will explicitly train and evaluate category and urgency classification on English, Roman Urdu, and code-switched complaints, alongside policy-grounded assistance.

3.3.3 Gorgias AI Agent

Gorgias offers an AI agent for e-commerce support, while its Shopify-connected helpdesk provides order context alongside customer conversations. [13] [14]

Its e-commerce specialization makes it a useful comparison for contextual customer assistance. The reviewed documentation does not establish performance on Resolve AI's Roman Urdu labels or Pakistani return/refund prediction target.

Resolve AI will use complaint text, available order details, previous interactions, and seller policies to support relevant responses. Actions offered by Gorgias do not extend Resolve AI's risk-scoring scope into refund processing.

3.3.4 Intercom Fin

Intercom documents Fin for Ecommerce as a Shopify-connected customer-assistance system using store information and customer or order context. [15]

The integration illustrates that store-grounded assistance already exists. Its documented e-commerce functions should not be mistaken for evidence of performance on this project's bilingual classifier or local risk model.

Fin provides a comparison for Shopify-connected contextual support. Resolve AI will evaluate its own English and Roman Urdu classification, policy-grounded suggestions, and return/refund-risk scoring.

3.3.5 PKAILabs

PKAILabs advertises a Pakistan-oriented WhatsApp assistant using business knowledge through RAG. It states that the assistant understands English, Urdu, and Roman Urdu and can hand conversations to staff. [16]

These advertised features overlap with Resolve AI's bilingual, policy-grounded complaint workflow. The page does not provide independent evaluation of language quality or document an order-level return/refund-risk model.

Resolve AI's distinction must therefore rely on its evaluated classification and integrated risk scoring, rather than a claim that local Roman Urdu assistance is unavailable.

3.3.6 ShuttlePro

ShuttlePro advertises a Pakistan-focused support platform with Shopify integration, complaint tickets, and WhatsApp workflows. Its Intelligence offering describes order-risk scoring and sentiment or urgency signals. [17]

The documented order-risk approach includes configurable rules and anomaly signals. The reviewed page does not establish that its score is the same as a calibrated return/refund-outcome probability, nor provide a comparable Roman Urdu evaluation.

ShuttlePro is a close local workflow comparison. Resolve AI must clearly define its risk target, complaint labels, and evaluation rather than claim that Pakistani platforms lack predictive or complaint-analysis capabilities.

3.4 Summary Tables

Table 3.1 compares the reviewed applications using the development-project format. Table 3.2 summarizes the seven essential research studies and their implementation relevance. The listed strengths and limitations are qualitative comparisons; no unperformed product tests or project accuracy results are reported.

Table 3.1: Related applications and their relevance to Resolve AI

Application	Features	Relevance	Limitations
Loop Returns [10]	Order-level return probabilities	Direct risk-scoring comparison	Limited enterprise pilot; local performance not established
Zendesk AI Agents [11]	Knowledge-based support; Urdu	Complaint assistance and language comparison	Transliterated language input unsupported
Gorgias AI Agent [13]	E-commerce AI support	Order-aware complaint assistance	Project-specific language and risk results not established
Intercom Fin [15]	Shopify-connected AI assistance	Contextual store support	Local risk and Roman Urdu evaluation not established
PKAILabs [16]	Roman Urdu; RAG; staff handoff	Local bilingual assistance comparison	Vendor claims; risk model not documented on reviewed page
ShuttlePro [17]	Order risk; sentiment; CRM workflow	Local support and scoring comparison	Risk target and comparable evaluation not established

Table 3.2: Research studies supporting Resolve AI development

Study and year	Technique	Relevance	Limitation
XGBoost (2016) [1]	Tabular tree boosting	Return-risk model	Data quality and tuning
LightGBM (2017) [2]	Efficient tree boosting	Alternative risk model	Requires local evaluation
Multilingual BERT Transfer (2019) [3]	Cross-lingual transfer	Bilingual model alternative	Transfer varies by language
XLM RoBERTa (2020) [4]	Multilingual representations	Primary classifier candidate	Roman Urdu requires testing
Sentence BERT (2019) [5]	Semantic sentence embeddings	Policy retrieval	Similarity is not eligibility

Study and year	Technique	Relevance	Limitation
Retrieval Augmented Generation (2020) [6]	Retrieval plus generation	Grounded response drafting	Retrieval and generation errors
XLM T (2022) [7]	Multilingual social-text modelling	Sentiment model candidate	Local language validation

The comparison supports a focus on integrated order-risk review and bilingual complaint assistance. Existing applications already demonstrate returns workflows and contextual support. Resolve AI's proposed value is bringing locally trained risk scoring, English and Roman Urdu complaint analysis, and seller-policy guidance into one working application. This value must be established through component evaluation and end-to-end workflow testing.

3.5 Conclusion

The reviewed applications provide references for return-risk scoring, case organization, bilingual assistance, contextual replies, and staff involvement. The selected studies provide established approaches for return/refund-risk prediction, bilingual complaint classification, sentiment analysis, semantic retrieval, and policy-grounded response generation. Resolve AI will apply these techniques within the original scope and evaluate their suitability for the intended data and users. The next chapter translates the project vision and these design considerations into software requirements.

Chapter 4 Software Requirement Specifications

This chapter specifies the expected behaviour of Resolve AI as a development project. It defines the features, user responsibilities, quality requirements, use cases, proposed interfaces, data structure, and risks that will guide implementation and testing. The return/refund module provides risk scores only. It does not approve returns, issue refunds, arrange exchanges, or manage return shipping. Complaint assistance and notifications are separate from financial transactions.

4.1 List of Features

The application will provide the following features within the agreed scope:

- Shopify order, return, and refund event ingestion and normalized records.
- Order-level return/refund-risk scores using order features and available prior customer history.
- English, Roman Urdu, and Urdu-English complaint category and urgency classification.
- Sentiment analysis as an additional complaint-handling signal.
- Seller-policy retrieval and contextual resolution suggestions.
- Complaint routing, staff assignment, escalation, and human takeover.
- Analytics for observed outcomes, risk tiers, complaint activity, and supported courier indicators.
- WhatsApp text complaint intake and eligible customer notifications.

Account access, store configuration, and policy maintenance are supporting functions required to make these features usable.

4.2 Functional Requirements

Each requirement has an identifier for use-case and design traceability. Owner, manager, and agent permissions are scoped to a store; customers communicate through WhatsApp rather than through a staff dashboard account.

4.2.1 Seller and Store Owner

FR-S01 The system shall allow a seller to register, sign in, and sign out using an application account.

FR-S02 The system shall allow the owner to authorize a Shopify store connection and inspect integration and synchronization status.

FR-S03 The system shall allow the owner to upload policy text or a supported text-based document, review indexing status, and activate a successfully indexed policy version.

FR-S04 The system shall display the latest valid return/refund-risk score, tier, scoring time, and model version for each scoreable order. Unavailable scores shall be explicitly labelled.

FR-S05 The owner shall be able to maintain store staff memberships and permitted complaint-routing settings. Financial or return-processing actions shall not be offered.

FR-S06 The system shall display date-filtered order-outcome, risk-tier, and complaint summaries for the owner's store. Courier indicators shall be displayed only when their required fields are available.

FR-S07 The owner shall be able to configure the store's WhatsApp connection and eligible notification settings without exposing secret credentials.

4.2.2 Store Manager and Support Agent

FR-A01 Staff shall access only stores for which they have an active membership and only actions permitted by their role.

FR-A02 Authorized staff shall be able to view complaint history, verified order context, category, urgency, sentiment, and processing status, and record category or urgency corrections.

FR-A03 The system shall generate a response suggestion using the complaint, relevant order context, and applicable seller-policy passages. It shall preserve the policy evidence used.

FR-A04 Authorized staff shall be able to edit and approve a reply before it is sent. The sent content and delivery state shall be retained.

FR-A05 Managers shall be able to assign or reassign cases; assigned agents shall be able to escalate, mark awaiting customer, or resolve complaints. Human takeover shall pause automatic replies.

4.2.3 Customer

FR-C01 Customers shall be able to submit English, Roman Urdu, or code-switched text complaints through the connected WhatsApp number.

FR-C02 The system shall ask for clarification when an order cannot be identified. It shall not reveal another customer's order details based only on a supplied order number.

FR-C03 Customers shall receive approved complaint replies and eligible order or return-status notifications through WhatsApp, subject to provider messaging rules and recorded permission where

required.

4.2.4 External Services and Automated Processing

Shopify webhooks provide event-driven updates to connected applications. [18] Event receipt and model work are separate operations so that ingestion does not depend on the time taken by inference.

FR-X01 The system shall verify provider webhook authenticity, persist necessary event data, and process duplicate deliveries without duplicating business records.

FR-X02 The system shall reconcile available Shopify records after missed or failed events and use event timestamps to avoid overwriting newer data with older updates.

FR-X03 The risk component shall return an estimate between 0 and 1 for supported inputs and record the feature snapshot and model version. Prior-history inputs shall exclude future outcomes.

FR-X04 The WhatsApp integration shall record incoming messages, associate them with the relevant conversation case, and deduplicate provider message identifiers.

FR-X05 The complaint pipeline shall predict category, urgency, and sentiment for incoming text and preserve model output separately from staff corrections.

FR-X06 Retrieval shall restrict policy passages to the relevant store and active policy version. Missing or conflicting evidence shall trigger review rather than an unsupported automatic answer.

FR-X07 Routing shall use defined urgency, confidence, sentiment, and relevant case-context conditions. Permitted routine replies may be automatic; high-urgency or uncertain cases shall be assigned for human review or escalation.

FR-X08 Notification delivery shall track queued, submitted, delivered, read, or failed states where reported by the provider, and shall not treat API acceptance as proof of delivery.

FR-X09 The system shall record significant policy, assignment, routing, staff-reply, and case-state changes in a store-scoped audit history.

4.3 Quality Attributes

Usability requires clear labels, visible processing states, and a complaint view that places messages, order context, and policy evidence together. Reliability requires durable event receipt, controlled retries, and continued manual access when an AI component is unavailable. Security requires store isolation and role-based access. Maintainability requires separate integration, inference, retrieval, and case-handling components. Traceability requires requirement identifiers, model versions, policy versions, and records

of staff decisions. These attributes are converted into verification conditions in the following section.

4.4 Non-Functional Requirements

The numerical values below are proposed acceptance targets for the evaluation environment, not measured results. The test environment shall document CPU, memory, network conditions, model configuration, and dataset size so that results can be interpreted.

Table 4.1: Non-functional requirements and verification conditions

ID	Requirement	Verification
NFR-01	At 20 concurrent authenticated dashboard users, 95 percent of cached or database-backed list and detail requests shall complete within 3 seconds, excluding external provider and LLM calls.	Load test a store with 10,000 order records and 1,000 cases; report errors and latency percentiles.
NFR-02	Valid webhooks shall be durably recorded and acknowledged within a target of 2 seconds at the 95th percentile in the evaluation environment; inference shall run afterwards.	Measure receipt-to-acknowledgement time and verify persistence before success acknowledgement.
NFR-03	With models loaded, risk inference shall target 2 seconds and complaint classification 5 seconds at the 95th percentile for a supported single input. Response generation shall show pending state and use a configurable 30-second timeout.	Measure component latency separately; disclose provider latency and timeout outcomes.
NFR-04	Repeated delivery of the same event or message shall create one logical record and at most one notification job for the same event and recipient.	Replay duplicate deliveries and check database constraints and job keys.
NFR-05	Temporary provider failures shall retain pending work, use bounded retries, and expose terminal failures for review.	Simulate outages and restart processing; verify recorded work is recoverable without duplicate effects.

ID	Requirement	Verification
NFR-06	Protected traffic shall use HTTPS; passwords shall be hashed, credentials kept outside public source, and store access checked server-side.	Test unauthorized and cross-store access and inspect credential handling.
NFR-07	New suggestions shall reference the active policy version; automatic replies shall not assert unsupported refunds, exchanges, or actions.	Test policy replacement, missing evidence, and misleading customer instructions.
NFR-08	Risk evaluation shall report precision, recall, F1, ROC-AUC and, where relevant, PR-AUC against a simple baseline. Bilingual classification shall report macro F1 and per-class results separately by language group.	Use held-out records and messages with documented splits; do not select models on the test set.
NFR-09	Representative users shall be able to find a risk score, inspect a complaint, and approve a reply after a brief orientation.	Conduct task-based usability checks and record completion, errors, and feedback.
NFR-10	Integration adapters and model interfaces shall allow a provider or checkpoint to be changed without redesigning case records.	Review interfaces and test a replacement adapter or mocked provider through the same contract.
NFR-11	Stored timestamps shall use a consistent UTC basis and display the configured local timezone. Backups shall support restoration of a captured database state.	Check timezone conversion and perform a restore exercise, including policy-index reconstruction.

4.5 Assumptions

- The seller can authorize Shopify access and provide a WhatsApp business connection with the permissions needed for the selected functions.
- The seller supplies accurate, current return and refund policies in supported text form.
- Customer-history features are used only where stable identifiers and prior records are available. Missing history is represented explicitly.

- English and Roman Urdu complaint examples can be labelled using a documented category and urgency scheme.
- A pretrained sentiment model and embedding model can be selected and evaluated for the intended language mix.
- Internet access is available for provider APIs and any externally hosted generation service.
- Courier performance is conditional on suitable courier and delivery fields in synchronized records; direct courier integrations are not assumed.
- Human staff remain available for uncertain or sensitive complaints and for actions outside permitted automatic replies.

4.6 Use Cases

The following tables describe actor actions and observable system responses. They cover the primary workflows and supporting access and configuration functions. References connect each use case to the functional requirements.

Table 4.2: UC-01 Create seller account

Name	UC-01 Create seller account
Actors	Seller
Summary	Register a store owner for access to Resolve AI.
Pre-Conditions	The seller is not signed in and has a valid email address.
Post-Conditions	An account is created; no Shopify connection is assumed.
Special Requirements	Duplicate accounts must be rejected.
Requirement Reference	FR-S01
Basic Flow	
Actor Action	**System Response**
1. Seller supplies name, email, and password.	System validates input and creates the account with a password hash.
2. Seller signs in.	System opens onboarding.
Alternative Flow	Existing email or invalid input: display validation feedback without creating a second account.

Table 4.3: UC-02 Sign in and sign out

Name	UC-02 Sign in and sign out
Actors	Seller, manager, support agent
Summary	Access the dashboard with the permissions assigned to the user.
Pre-Conditions	An active account and, for staff, a store membership exist.
Post-Conditions	A session is established; signing out invalidates that session.
Special Requirements	Authentication errors must not reveal whether a particular account exists.
Requirement Reference	FR-S01; FR-A01
Basic Flow	
Actor Action	**System Response**
1. User enters email and password.	System verifies credentials and membership.
2. User signs out.	System invalidates the session and returns to sign in.
Alternative Flow	Invalid credentials or inactive account: deny access and show a general error.

Table 4.4: UC-03 Connect Shopify store

Name	UC-03 Connect Shopify store
Actors	Seller
Summary	Authorize the application to read the store data needed for scoring and complaint context.
Pre-Conditions	The seller is signed in and can authorize the Shopify store.
Post-Conditions	The store connection and required event subscriptions are recorded.
Special Requirements	Credentials must not be shown in the dashboard or logs.
Requirement Reference	FR-S02
Basic Flow	
Actor Action	**System Response**
1. Seller selects Connect Shopify and authorizes access.	System validates the authorization and associates the store.

2. Seller returns to integration settings.	System displays connection and synchronization status.
Alternative Flow	Authorization declined or insufficient access: keep the connection inactive and explain the next step.

Table 4.5: UC-04 Synchronize order outcomes

Name	UC-04 Synchronize order outcomes
Actors	Shopify; seller
Summary	Receive order, return, and refund events for application records.
Pre-Conditions	The Shopify integration and webhook subscriptions are active.
Post-Conditions	Valid events are recorded once and associated order records are updated.
Special Requirements	Verify authenticity, deduplicate events, and handle events arriving out of order.
Requirement Reference	FR-X01; FR-X02
Basic Flow	
Actor Action	**System Response**
1. Shopify submits an event.	System verifies and durably records the event before acknowledging receipt.
2. Seller reviews synchronization status.	System shows the latest successful update and unresolved failures.
Alternative Flow	Duplicate event: acknowledge without another update. Invalid event: reject. Missing events: use reconciliation.

Table 4.6: UC-05 Review return/refund-risk score

Name	UC-05 Review return/refund-risk score
Actors	Seller, manager
Summary	Inspect an order's predicted risk using available order and prior customer information.
Pre-Conditions	An order is synchronized and a compatible risk model is available.
Post-Conditions	The score, tier, model version, and scoring time are visible.
Special Requirements	A score must never initiate a refund or establish fraud.

Requirement Reference	FR-S04; FR-X03
Basic Flow	
Actor Action	**System Response**
1. User opens an order or filters the risk list.	System retrieves the latest valid score and related context.
2. User reviews the score.	System shows a value between 0 and 1 and the defined risk tier.
Alternative Flow	Missing history: use the supported missing-data path. Invalid features or unavailable model: display score unavailable.

Table 4.7: UC-06 Maintain seller policies

Name	UC-06 Maintain seller policies
Actors	Seller
Summary	Upload or replace return/refund policy text used for response suggestions.
Pre-Conditions	The seller is authenticated for the store.
Post-Conditions	A policy version is saved and searchable after successful indexing.
Special Requirements	Only the current approved policy version may be used for new suggestions.
Requirement Reference	FR-S03; FR-X06
Basic Flow	
Actor Action	**System Response**
1. Seller submits policy text or a supported text-based document.	System validates and stores a new version.
2. Seller reviews indexing status.	System displays Ready or an actionable error after preparing policy passages.
Alternative Flow	Empty or unreadable document: reject it. Indexing failure: retain the previous active version.

Table 4.8: UC-07 Submit a WhatsApp complaint

Name	UC-07 Submit a WhatsApp complaint

Actors	Customer; WhatsApp service
Summary	Create or continue a text complaint conversation.
Pre-Conditions	The store's WhatsApp connection is active.
Post-Conditions	A message is recorded under a complaint case and processing is queued.
Special Requirements	Duplicate messages must not create duplicate complaints.
Requirement Reference	FR-C01; FR-C02; FR-X04
Basic Flow	
Actor Action	**System Response**
1. Customer sends a text complaint, optionally including an order reference.	System records the message and finds or creates the conversation case.
2. Customer provides a requested order reference.	System links the order only after checking customer ownership.
Alternative Flow	Unknown or ambiguous order: request clarification. Unsupported attachment: request a text description.

Table 4.9: UC-08 Review complaint analysis

Name	UC-08 Review complaint analysis
Actors	Support agent, manager
Summary	Inspect predicted category, urgency, and sentiment.
Pre-Conditions	A case exists with at least one incoming text message.
Post-Conditions	Analysis is displayed and authorized corrections are saved.
Special Requirements	The original model output must remain distinct from a staff correction.
Requirement Reference	FR-A02; FR-X05
Basic Flow	
Actor Action	**System Response**
1. Agent opens a complaint.	System displays message history and available analysis.
2. Agent corrects a category or urgency.	System records the correction and actor without overwriting model evidence.

Alternative Flow	Analysis failure: display pending or failed status and keep the case available for manual handling.

Table 4.10: UC-09 Review and send response

Name	UC-09 Review and send response
Actors	Support agent
Summary	Use policy-grounded suggestions to reply to the customer.
Pre-Conditions	The case is accessible to the agent and a messaging connection is available.
Post-Conditions	The approved response and delivery state are recorded.
Special Requirements	Suggestions must not claim that a refund or exchange has been performed.
Requirement Reference	FR-A03; FR-A04; FR-X06
Basic Flow	
Actor Action	**System Response**
1. Agent requests a suggestion.	System retrieves policy passages and drafts a contextual response.
2. Agent edits and approves the draft.	System queues the approved message and records its sender.
Alternative Flow	No policy evidence or contradictory information: flag for manual response. Delivery failure: retain reply and expose retry status.

Table 4.11: UC-10 Route and resolve a complaint

Name	UC-10 Route and resolve a complaint
Actors	Manager, support agent; routing service
Summary	Assign a case to automatic response, human review, or escalation.
Pre-Conditions	The case exists and the store's permitted handling conditions are configured.
Post-Conditions	The routing decision, owner where relevant, and case state are recorded.
Special Requirements	Human takeover pauses automated responses; risk scoring never authorizes financial actions.

Requirement Reference	FR-S05; FR-A05; FR-X07
Basic Flow	
Actor Action	**System Response**
1. New analysis becomes available or manager assigns the case.	System selects or updates the handling route.
2. Assigned agent resolves or escalates the complaint.	System records the transition and maintains the conversation history.
Alternative Flow	High urgency, uncertain analysis, or unsupported policy answer: send to human review. New customer follow-up can reopen a closed case.

Table 4.12: UC-11 Receive customer notifications

Name	UC-11 Receive customer notifications
Actors	Customer; WhatsApp service
Summary	Communicate eligible order, return-status, or complaint updates.
Pre-Conditions	An eligible event, valid recipient, and permitted messaging route exist.
Post-Conditions	The notification is recorded with its submission or delivery state.
Special Requirements	Apply consent, template, and conversation rules required by the provider; avoid duplicate sends.
Requirement Reference	FR-C03; FR-X08
Basic Flow	
Actor Action	**System Response**
1. An eligible update is recorded.	System creates one notification for that event and recipient.
2. WhatsApp returns submission or delivery information.	System updates the notification state.
Alternative Flow	Unavailable recipient or provider failure: record the cause and retry only eligible transient failures.

Table 4.13: UC-12 Review analytics

Name	UC-12 Review analytics
Actors	Seller, manager
Summary	Inspect aggregated return outcomes, risk tiers, and complaint activity.
Pre-Conditions	The user has access to a connected store.
Post-Conditions	A filtered dashboard is displayed with data freshness information.
Special Requirements	Missing courier data must not be represented as zero performance.
Requirement Reference	FR-S06
Basic Flow	
Actor Action	**System Response**
1. User selects dates and relevant filters.	System displays the available metrics and charts.
2. User changes filters.	System recalculates the view for the authorized store.
Alternative Flow	No matching data: show an empty state. Unsupported courier fields: show unavailable metrics.

4.7 Hardware and Software Requirements

The following resources are proposed for implementation and evaluation. Framework and storage choices are design selections for the report, not statements that the application has already been implemented. Dependency versions will be pinned when the implementation environment is established.

4.7.1 Hardware Requirements

A development workstation should provide a modern multi-core processor, 16 GB RAM, and approximately 30 GB of available storage for code, datasets, environments, and model artefacts. Transformer fine-tuning should use a suitable GPU environment, with approximately 16 GB GPU memory as a planning allocation; actual needs depend on the checkpoint, sequence length, and batch configuration. Hosted GPU access may be used rather than requiring each team member to own such hardware. An initial application deployment may use 4 CPU cores and 8 GB RAM for the API, database, and worker, with inference hosted separately if needed. These allocations must be checked against measured load. End users require an internet-connected computer or phone with a supported browser; customers require WhatsApp access.

4.7.2 Software Requirements

Table 4.14: Proposed software components

Component	Selection	Purpose
Web interface	React with TypeScript	Staff dashboard and configuration screens.
Application API	Python with FastAPI	Access checks, cases, integrations, and API contracts.
Transactional storage	PostgreSQL	Store records, orders, cases, messages, and audit data.
Background work	Redis and a Python task worker	Deferred ingestion, inference, indexing, and delivery retries.
Risk modelling	XGBoost or LightGBM; pandas and scikit-learn	Data preparation, training, and evaluation.
Language inference	PyTorch and Transformers	Fine-tuned classifier and pre-trained sentiment model.
Policy retrieval	Sentence-transformers and Qdrant	Embeddings and seller-scoped policy search.
Response generation	An LLM accessed through a replaceable adapter	Grounded draft generation; model selected after evaluation.
Commerce connection	Shopify Admin API and webhooks	Authorized order and outcome synchronization.
Messaging	WhatsApp Cloud API	Complaint messages, replies, and notifications.
Development tools	Git, automated API and unit checks, and browser testing	Version control and repeatable verification.
Deployment	Containerized services and HTTPS endpoint	Reproducible service configuration and provider callbacks.

FastAPI is a Python API framework that supports typed request validation and API documentation. [19] The detailed division of responsibilities will be described in Chapter 6.

4.8 Graphical User Interface

The following screenshots show the implemented React frontend of Resolve AI using sample store, order, and complaint data. They demonstrate interface layout and navigation; model outputs, connection states, and conversations shown here are illustrative. Backend integration and measured system performance will be evaluated separately.

The seller navigation is Sign in → Settings and integrations → Overview, with access to Orders and Risk, Complaints, Policy Library, and Team settings. The agent workflow is Sign in → Complaint queue → Case detail → Review reply, escalate, or resolve. Customers communicate through WhatsApp rather than a separate web dashboard.

4.8.1 Account Access

Figures 4.1 and 4.2 show the sign-in and registration screens for sellers and staff. The forms include account details, password visibility controls, and validation feedback, supporting UC-01 and UC-02. Sign-out is available beside the workspace profile.

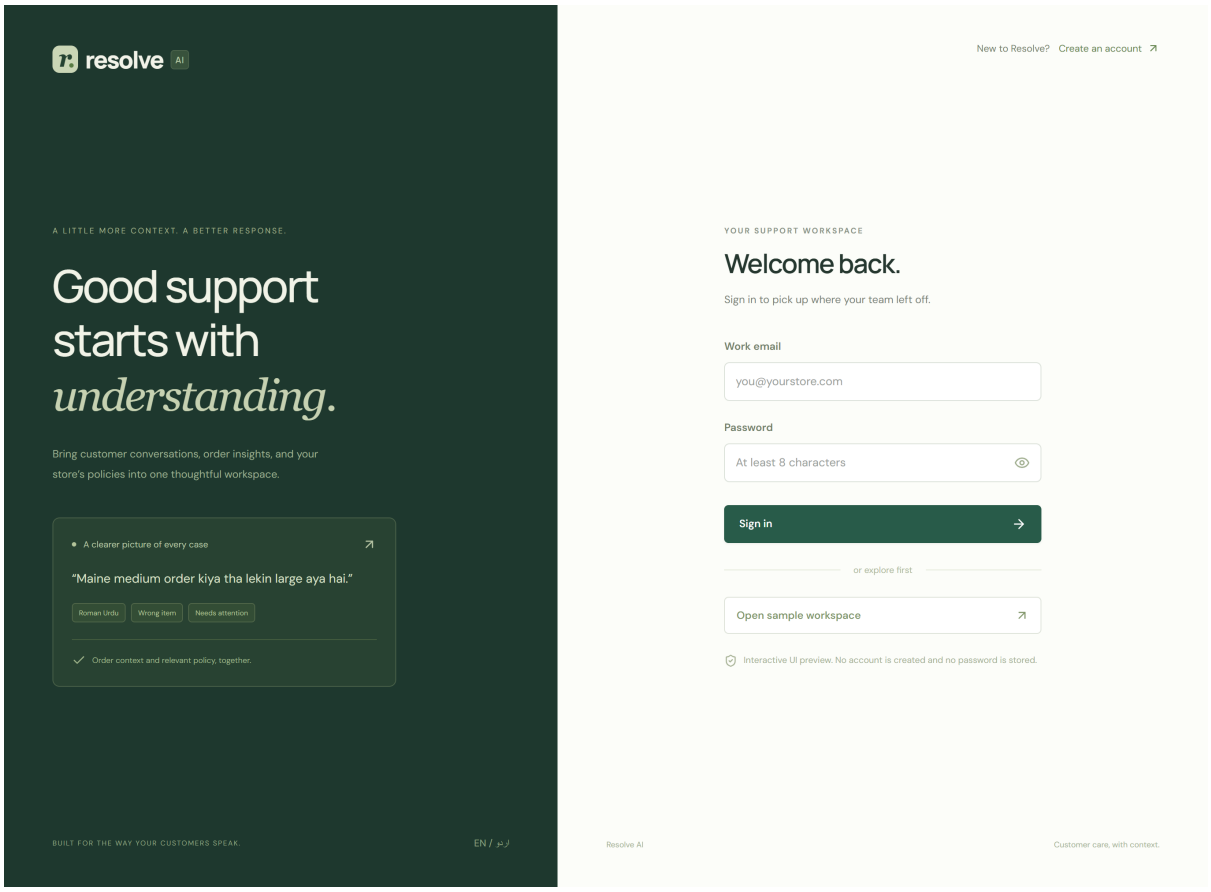


Figure 4.1: Sign-in screen for sellers and staff showing email and password fields (UC-02)

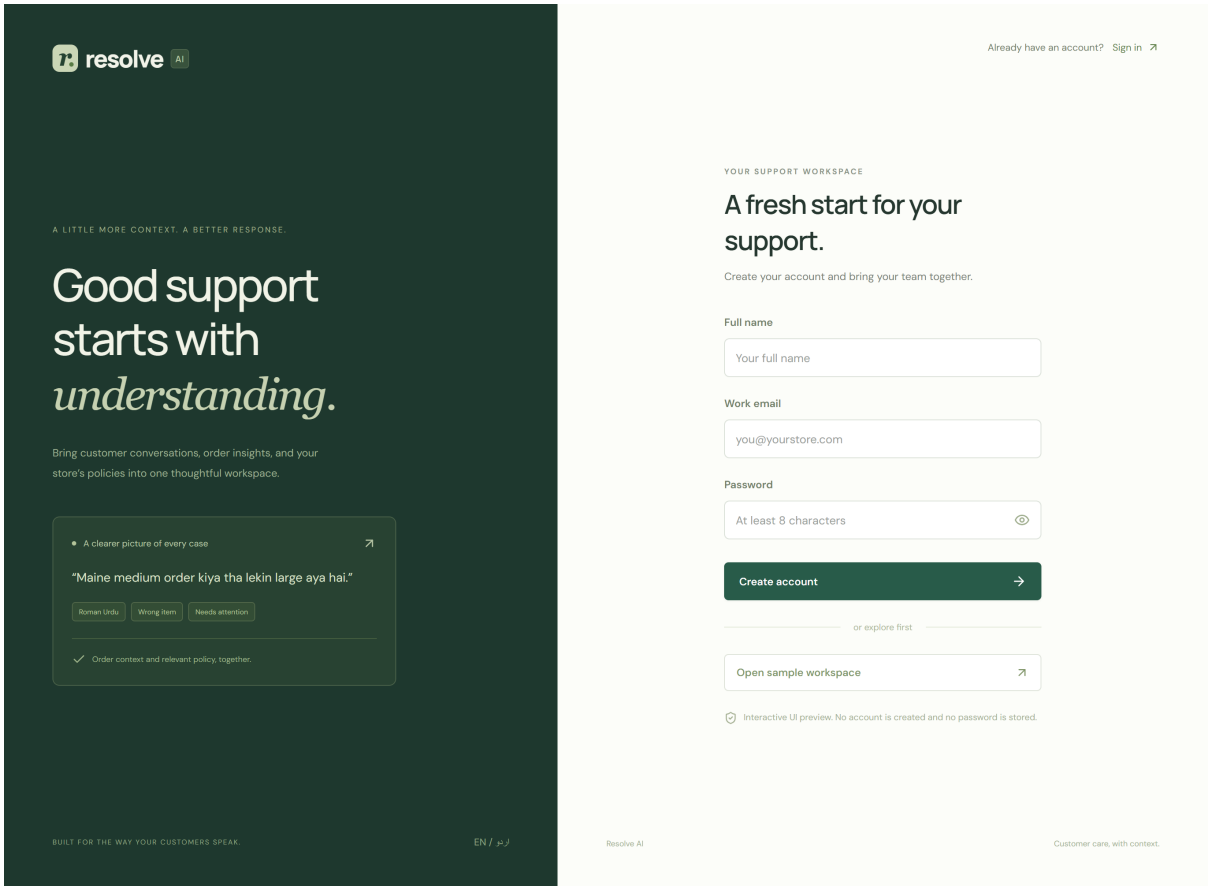


Figure 4.2: Account registration screen showing name, email, and password fields (UC-01)

4.8.2 Store, Messaging, and Team Settings

The integration screen in Figure 4.3 presents the Shopify store, synchronization status, and WhatsApp connection fields for UC-03 and UC-04. Figures 4.4 and 4.5 show supporting controls for team roles, access, and complaint-routing conditions.

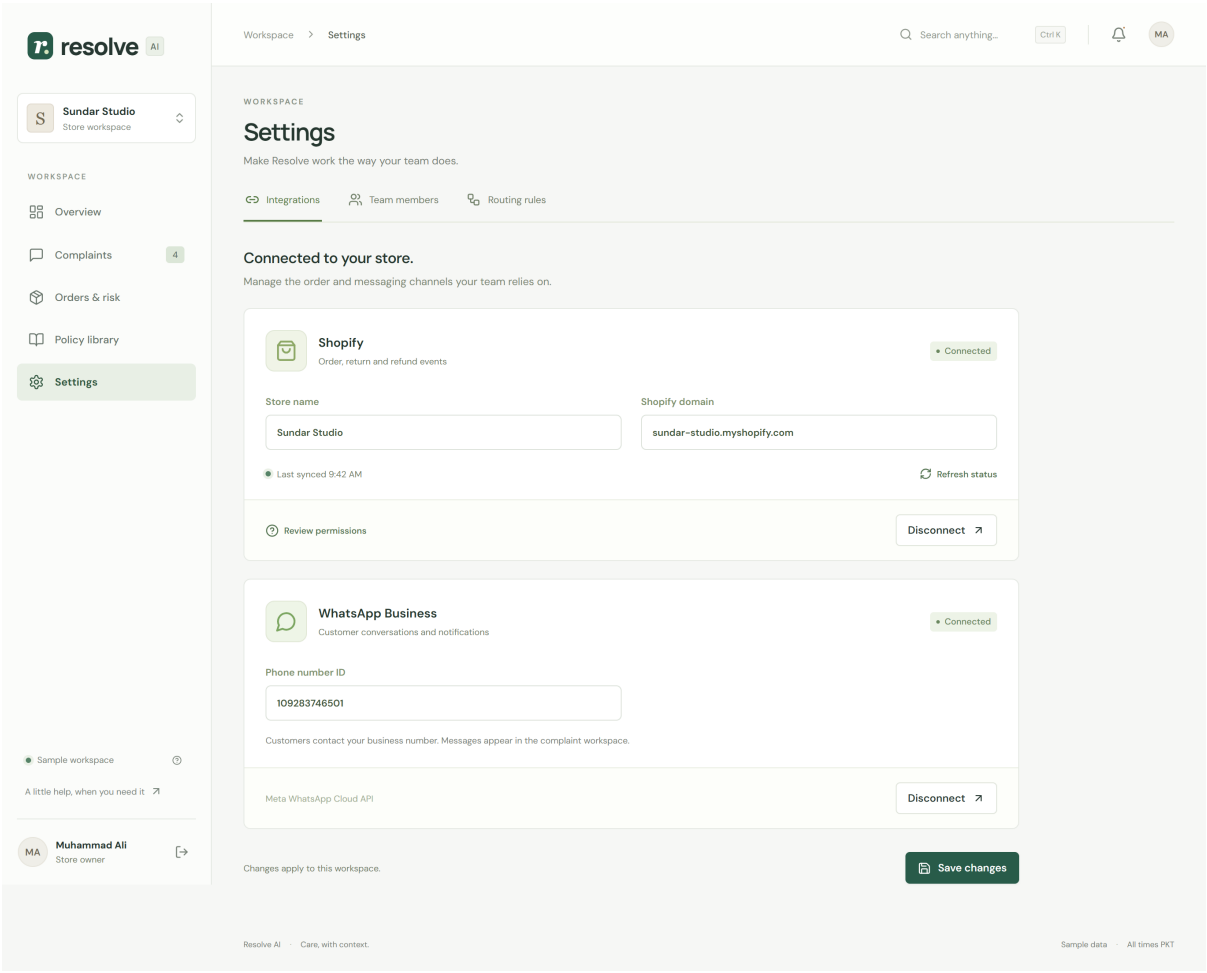


Figure 4.3: Store and messaging settings showing Shopify and WhatsApp connection configuration (UC-03 and UC-04)

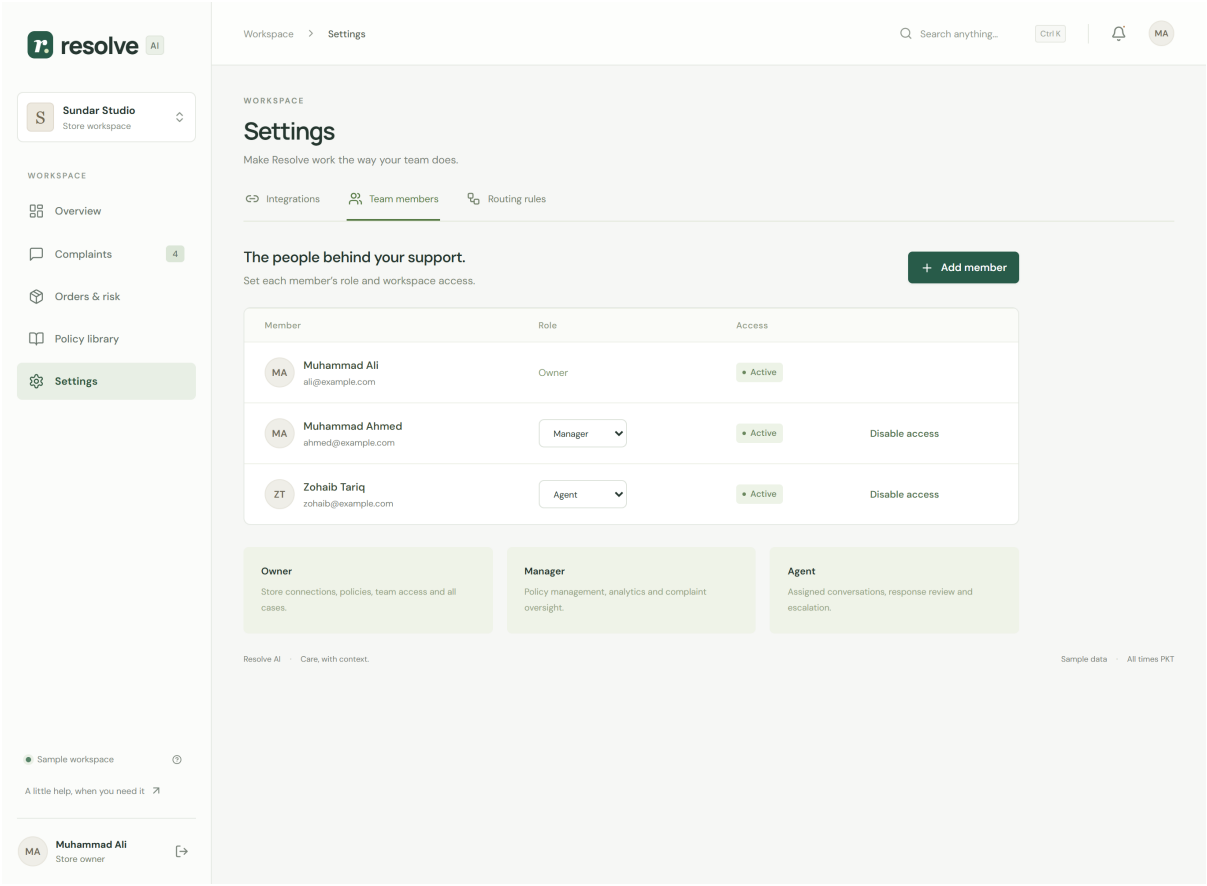


Figure 4.4: Team-management screen showing staff memberships, roles, and access controls

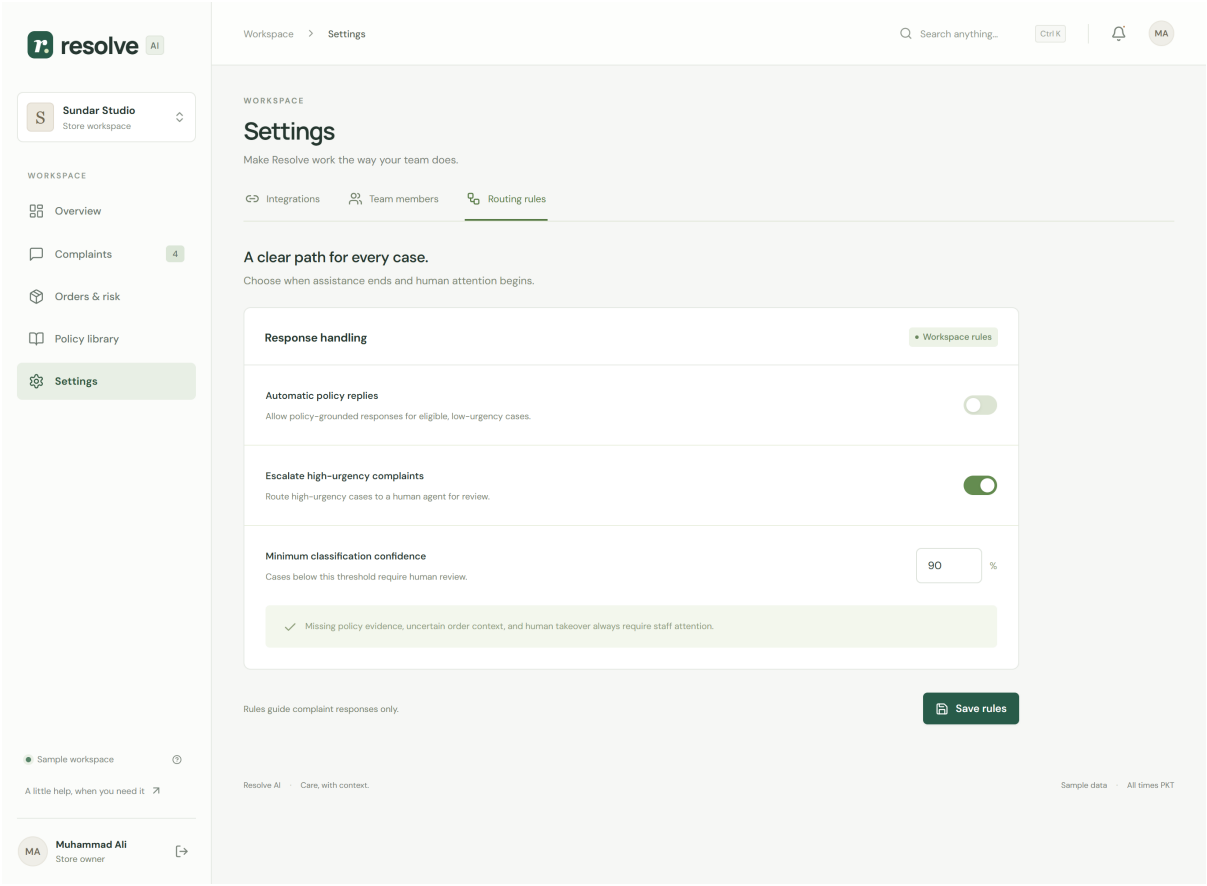


Figure 4.5: Complaint-routing settings showing automatic-reply, urgency, and confidence controls

4.8.3 Analytics Dashboard

Figure 4.6 presents the overview cards, order activity, risk distribution, and priority conversations. Figure 4.7 shows the remaining dashboard panels: flagged orders, complaint categories, observed order outcomes, and available courier coverage. The date selector and export control support UC-12.

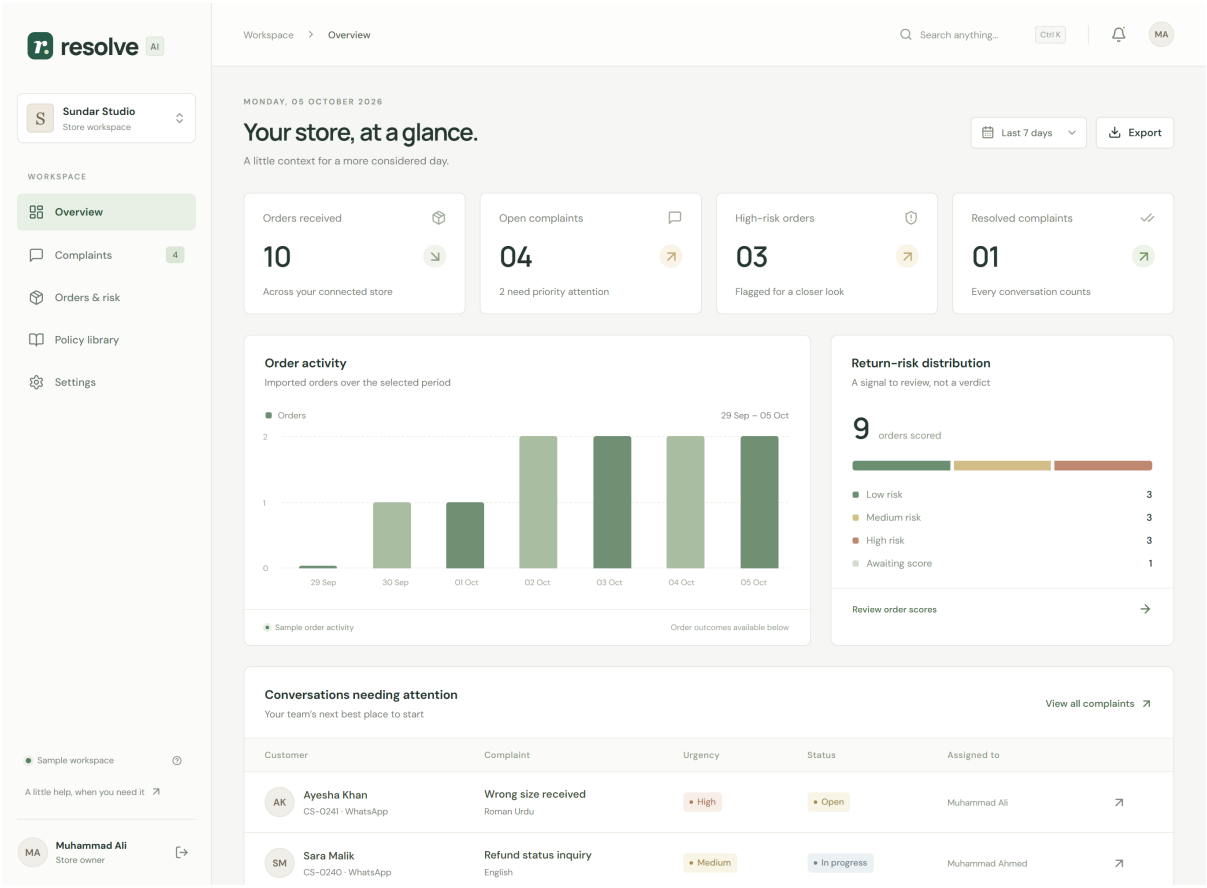


Figure 4.6: Analytics overview showing order counts, risk distribution, and priority conversations (UC-12)

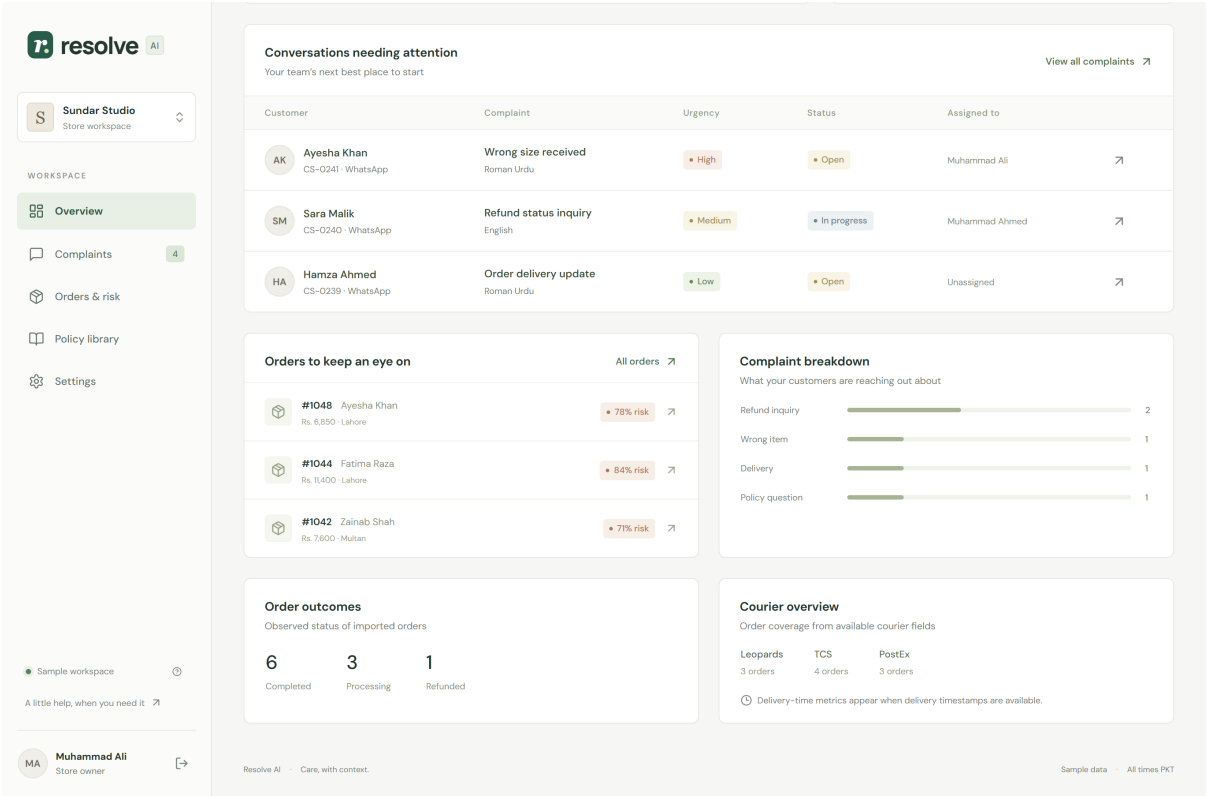


Figure 4.7: Additional dashboard panels showing flagged orders, complaint categories, order outcomes, and courier coverage (UC-12)

4.8.4 Orders and Return/Refund Risk

The order list in Figure 4.8 supports search and filtering by date and risk tier. Selecting an order opens the detail panel in Figure 4.9, which displays the score, available customer history, order information, and prediction metadata for UC-05. These controls support risk review.

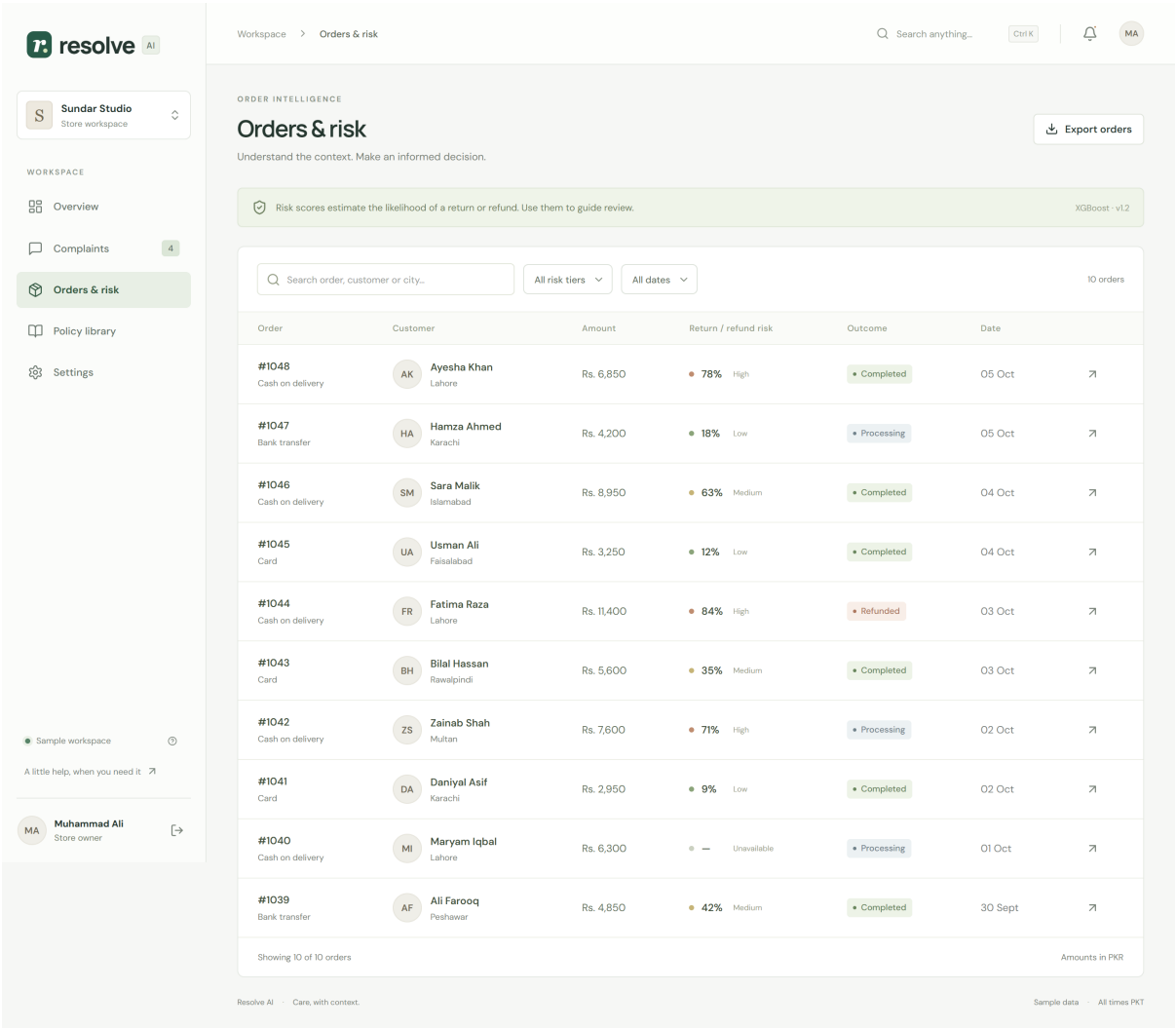


Figure 4.8: Order-risk list showing customer records, scores, tiers, and observed outcomes (UC-05)

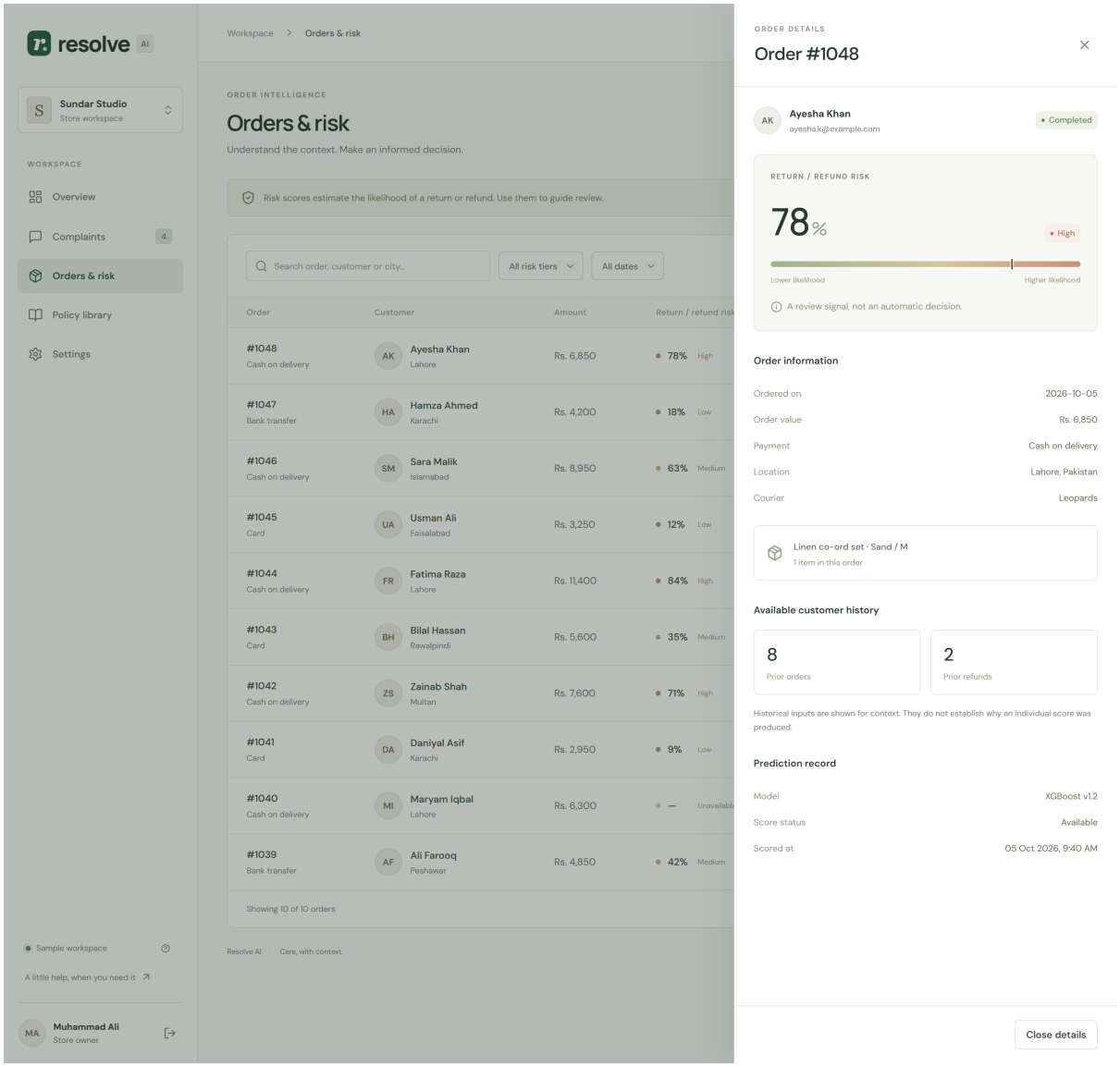


Figure 4.9: Selected-order detail panel showing risk score, customer history, and prediction metadata (UC-05)

4.8.5 Complaint Queue and Case Detail

Figure 4.10 combines the complaint queue, conversation history, editable response draft, classification results, and policy evidence. Staff can assign a case, enable human takeover, escalate, or mark it resolved. The illustrated Roman Urdu conversation covers the staff-facing workflows associated with UC-07, UC-08, UC-09, and UC-10.

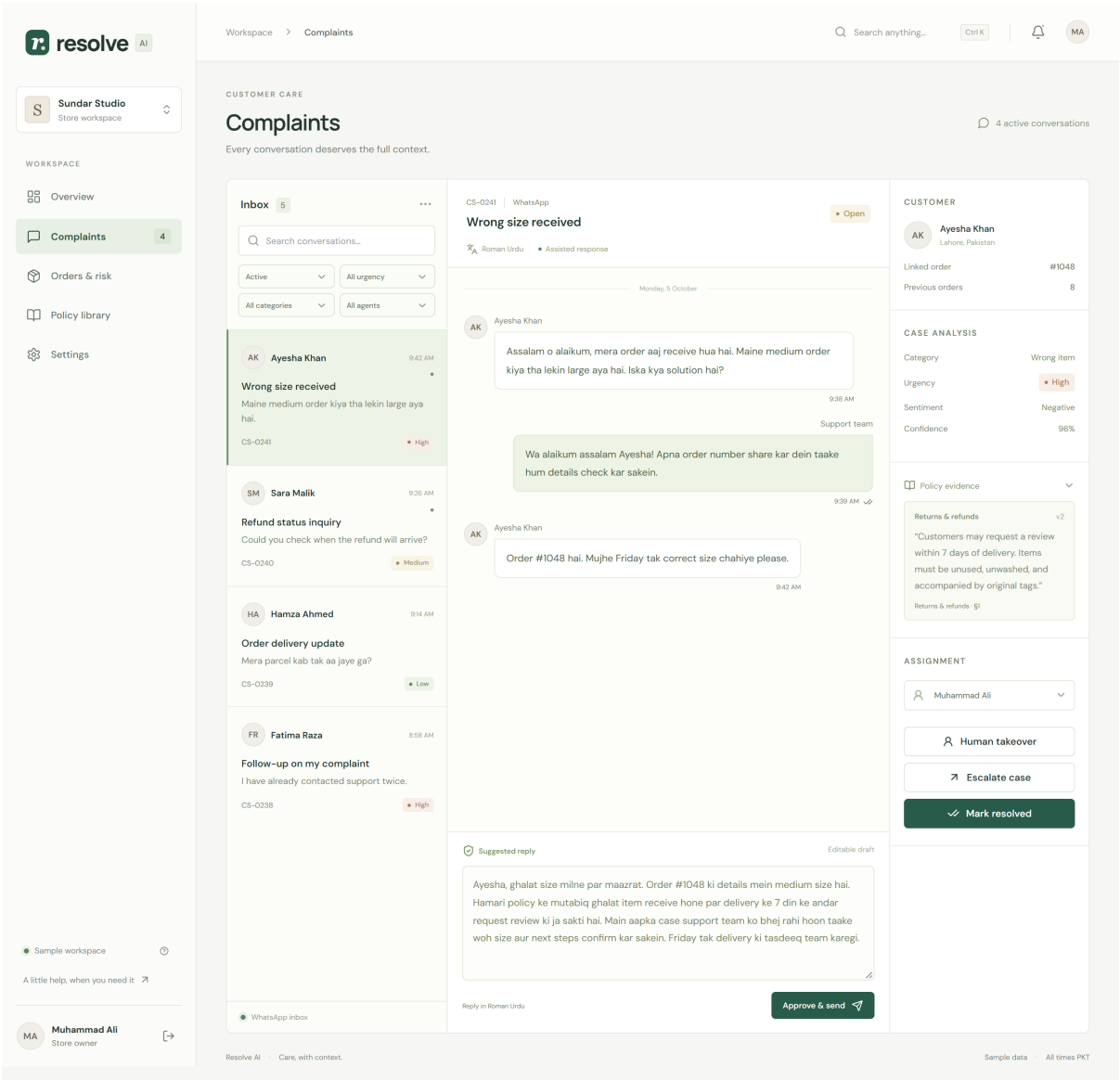


Figure 4.10: Complaint workspace showing a Roman Urdu conversation, analysis, policy evidence, reply editor, and case actions (UC-07 to UC-10)

4.8.6 Seller Policy Library

Figure 4.11 shows the policy editor, text-import control, version history, and save and activation actions. Staff select a document, review or edit its content, save a new version, and activate an eligible version as described in UC-06.

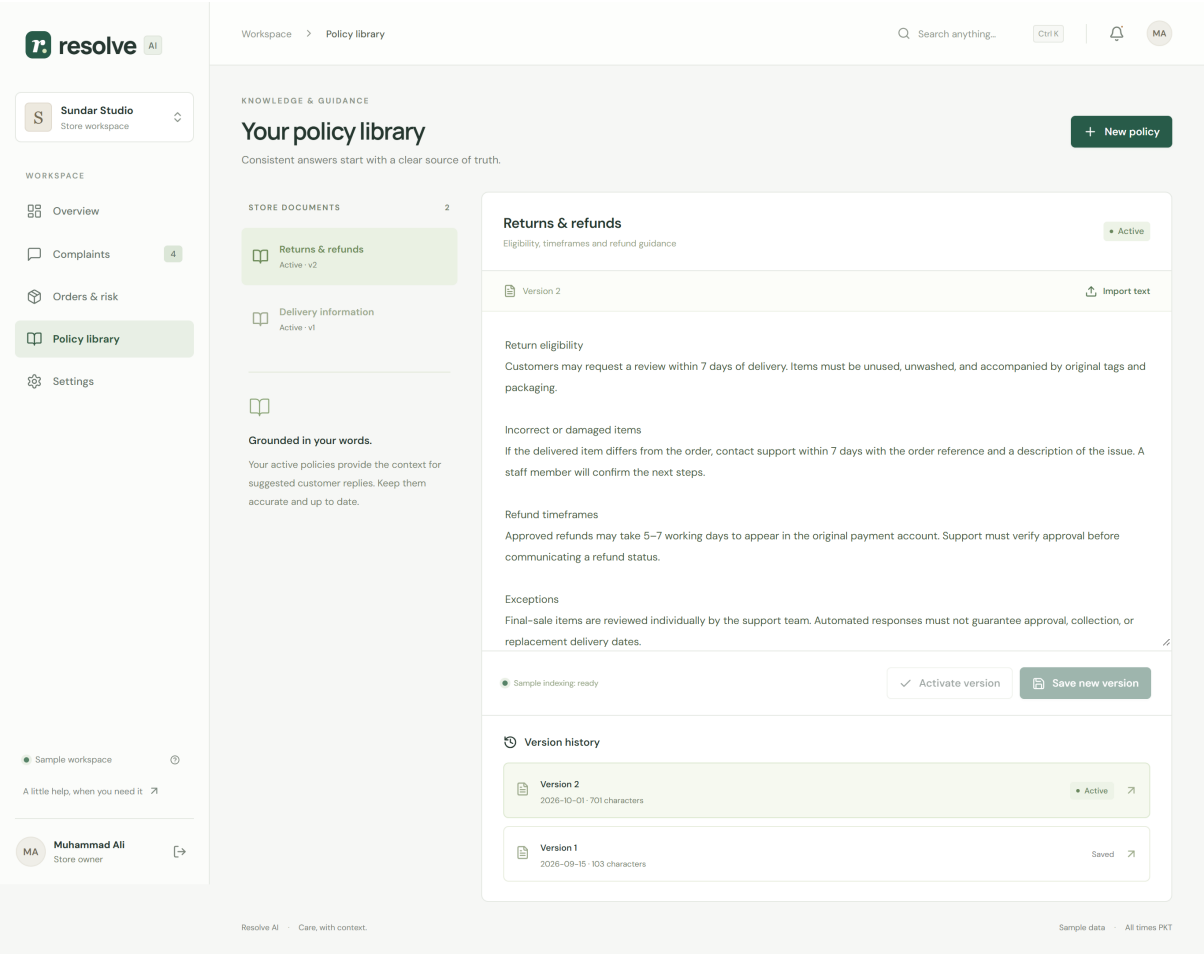


Figure 4.11: Seller policy library showing policy content, version history, and activation controls (UC-06)

4.9 Database Design

PostgreSQL is proposed for transactional data and Qdrant for policy embeddings. Each business record belongs to a store either directly or through its parent entity. The database stores observations and risk predictions; it does not contain a workflow for executing refunds or shipping returns. RiskScore records are versioned predictions, not changes to a Shopify order's financial state.

4.9.1 ER Diagram

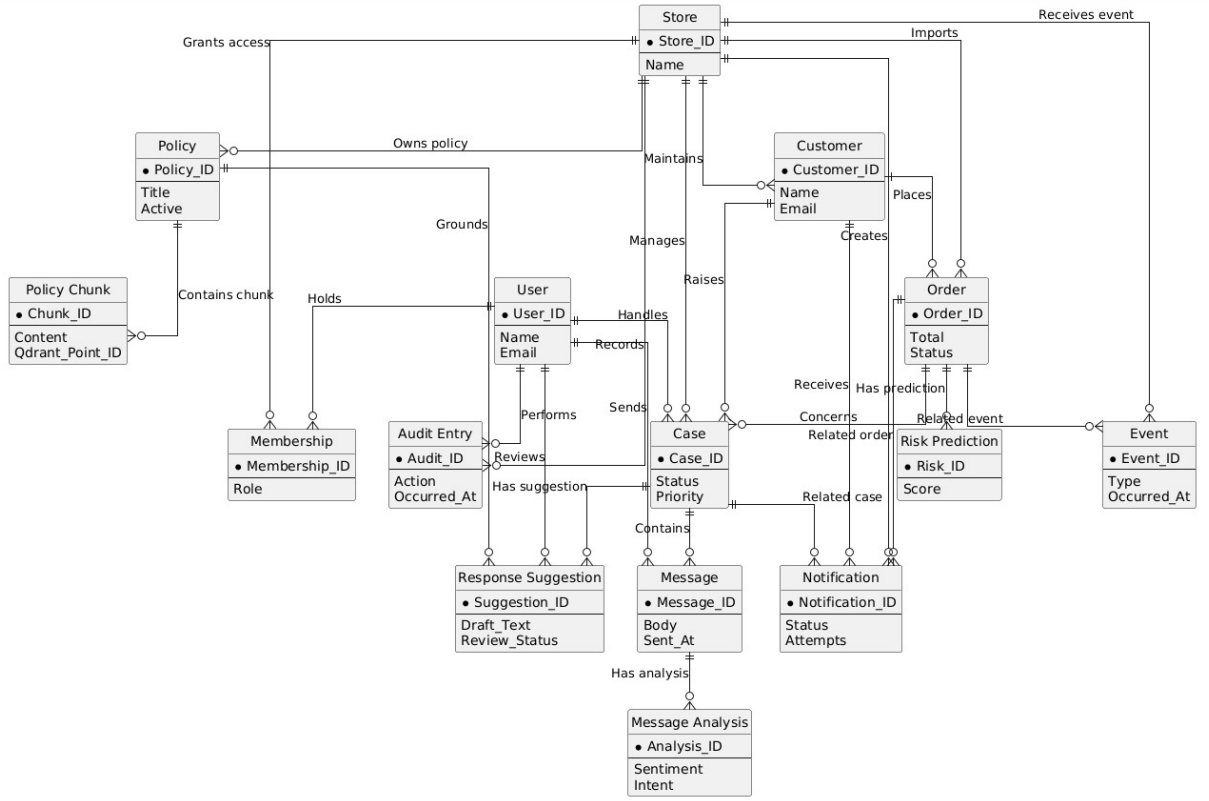


Figure 4.12: Entity Relationship Diagram of Resolve AI

The database entities cover store access, customer orders, return/refund-risk predictions, complaints, messages, policy knowledge, notifications, and audit records. Detailed field types and constraints are provided in the data dictionary.

A ComplaintCase can optionally link to a Customer, Order, and assigned User; a Store owns its cases, customers, policies, notifications, and audit entries. Notifications can refer to an order, a case, or both. A suggestion stores retrieved chunk identifiers as evidence, and its policy-version reference identifies the source used. The application verifies all such references belong to the same store. Qdrant points carry the store, policy version, and chunk identifiers for filtering.

4.9.2 Data Dictionary

The following tables describe each field using a simple data type and a description. Types include String, Integer, Float, Decimal, Boolean, Date/Time, Categorical, and JSON. Primary keys, foreign keys, unique values, and optional fields are identified in the descriptions. Internal identifiers are UUIDs represented as strings; date and time values use UTC. Decimal is used for monetary amounts to preserve precision. Unless described as optional, fields are required.

Table 4.15: Store data dictionary

Field	Type	Description
id	String	Store identifier. Primary key.
name	String	Seller-facing store name.
shopify_domain	String	Authorized Shopify domain. Unique value.
shopify_token_ref	String	Reference to encrypted credential storage.
whatsapp_number_id	String	Connected WhatsApp business phone identifier. Optional field.
whatsapp_token_ref	String	Reference to messaging credentials. Optional field.
connection_status	Categorical	Disconnected, active, or error.
last_synced_at	Date/Time	Latest completed synchronization. Optional field.

Table 4.16: User data dictionary

Field	Type	Description
id	String	Application user identifier. Primary key.
name	String	Staff or seller display name.
email	String	Sign-in address. Unique value.
password_hash	String	Secure hash; never a plain password.
active	Boolean	Whether access is permitted.
created_at	Date/Time	Account creation time.

Table 4.17: Membership data dictionary

Field	Type	Description
id	String	Store membership identifier. Primary key.
store_id	String	Associated Store. Foreign key.
user_id	String	Associated User. Foreign key.
role	Categorical	Owner, manager, or agent.
active	Boolean	Whether this membership permits access.

Table 4.18: Customer data dictionary

Field	Type	Description
id	String	Internal customer identifier. Primary key.
store_id	String	Store owning the record. Foreign key.
shopify_customer_id	String	External customer identifier. Optional field.
name	String	Name when supplied. Optional field.
phone	String	Normalized customer phone. Optional field.
email	String	Email when supplied. Optional field.
messaging_consent	Boolean	Recorded permission for eligible proactive updates.

Table 4.19: Order data dictionary

Field	Type	Description
id	String	Internal order identifier. Primary key.
store_id	String	Associated Store. Foreign key.
customer_id	String	Linked Customer if identified. Foreign key. Optional field.
shopify_order_id	String	Shopify order identifier, unique per store.
ordered_at	Date/Time	Time the order was placed.
total_amount	Decimal	Order total in the recorded currency.
currency	String	Currency code.
payment_method	String	Available payment-method category. Optional field.
items	JSON	Product identifiers, quantities, prices, and categories when supplied.
location	JSON	Available city or region; excluded from public displays. Optional field.
outcome_status	String	Latest synchronized order outcome.
delivery_metadata	JSON	Available courier, tracking, and delivery timestamps. Optional field.

Table 4.20: SourceEvent data dictionary

Field	Type	Description
id	String	Durable ingestion identifier. Primary key.
store_id	String	Associated Store. Foreign key.
order_id	String	Related Order when identifiable. Foreign key. Optional field.
provider_event_id	String	Provider identifier, unique within store and source.
source	Categorical	Shopify or WhatsApp.
event_type	String	Incoming topic or callback type.
payload	JSON	Necessary validated event details.
occurred_at	Date/Time	Provider event time.
processing_status	Categorical	Pending, processed, or failed.
attempts	Integer	Processing attempt count.

Table 4.21: RiskScore data dictionary

Field	Type	Description
id	String	Prediction identifier. Primary key.
order_id	String	Order being scored. Foreign key.
score	Float	Estimated risk in the interval 0 to 1.
tier	Categorical	Low, medium, or high according to documented thresholds.
model_version	String	Version of model and feature contract.
features_snapshot	JSON	Prediction-time inputs, including available prior-history aggregates.
scored_at	Date/Time	Prediction time.
valid	Boolean	Whether score remains applicable to the current inputs.

Table 4.22: ComplaintCase data dictionary

Field	Type	Description
id	String	Complaint identifier. Primary key.

Field	Type	Description
store_id	String	Store owning the case. Foreign key.
customer_id	String	Linked Customer when known. Foreign key. Optional field.
order_id	String	Order after ownership verification. Foreign key. Optional field.
assigned_user_id	String	Responsible staff member. Foreign key. Optional field.
status	Categorical	Open, awaiting customer, in progress, escalated, or resolved.
route	Categorical	Automatic response, human review, or escalation.
routing_reason	String	Explanation of selected route.
human_takeover	Boolean	Whether automated replies are paused.
category_override	String	Authorized staff correction. Optional field.
urgency_override	String	Authorized staff correction. Optional field.
created_at	Date/Time	Case creation time.

Table 4.23: Message data dictionary

Field	Type	Description
id	String	Conversation-message identifier. Primary key.
case_id	String	Related ComplaintCase. Foreign key.
provider_message_id	String	Provider reference for deduplication. Optional field.
sender_user_id	String	Staff sender for an approved reply. Foreign key. Optional field.
direction	Categorical	Incoming or outgoing.
body	String	Complaint or approved response text.
sent_at	Date/Time	Message time.
delivery_status	Categorical	Received, queued, submitted, delivered, read, or failed.

Table 4.24: ComplaintAnalysis data dictionary

Field	Type	Description
id	String	Analysis identifier. Primary key.
message_id	String	Incoming Message analysed. Foreign key.
category	String	Predicted issue class.
urgency	Categorical	Low, medium, or high.
sentiment	Categorical	Positive, neutral, or negative.
confidence	JSON	Scores associated with the predicted labels.
model_versions	JSON	Classifier and sentiment-model versions.
analysed_at	Date/Time	Analysis completion time.

Table 4.25: PolicyDocument data dictionary

Field	Type	Description
id	String	Policy-version identifier. Primary key.
store_id	String	Seller owning the policy. Foreign key.
version	Integer	Increasing version within the store.
title	String	Policy name.
content	String	Extracted source text.
index_status	Categorical	Pending, ready, or failed.
active	Boolean	Approved current policy version.
uploaded_at	Date/Time	Version creation time.

Table 4.26: PolicyChunk data dictionary

Field	Type	Description
id	String	Policy-passage identifier. Primary key.
document_id	String	Related PolicyDocument. Foreign key.
chunk_index	Integer	Passage order within the document.
content	String	Passage text used for retrieval.
qdrant_point_id	String	Corresponding vector point. Unique value.
embedding_version	String	Embedding-model identifier.

Table 4.27: ResponseSuggestion data dictionary

Field	Type	Description
id	String	Draft-response identifier. Primary key.
case_id	String	Related ComplaintCase. Foreign key.
draft_text	String	Generated response awaiting review or permitted automatic handling.
policy_document_id	String	Policy version used. Foreign key. Optional field.
evidence_chunk_ids	JSON	Retrieved PolicyChunk identifiers.
generation_model	String	LLM identifier.
status	Categorical	Pending review, approved, rejected, or expired.
reviewed_by	String	User who reviewed the suggestion. Foreign key. Optional field.
created_at	Date/Time	Generation time.

Table 4.28: Notification data dictionary

Field	Type	Description
id	String	Notification identifier. Primary key.
store_id	String	Associated Store. Foreign key.
customer_id	String	Intended Customer. Foreign key.
order_id	String	Order linked to the update. Foreign key. Optional field.
case_id	String	Complaint linked to the update. Foreign key. Optional field.
deduplication_key	String	Event and recipient key to avoid repeated sends. Unique value.
provider_message_id	String	Provider submission reference. Optional field.
payload	JSON	Permitted notification content and template reference.
status	Categorical	Queued, submitted, delivered, read, or failed.

Field	Type	Description
attempts	Integer	Send attempt count.

Table 4.29: AuditEntry data dictionary

Field	Type	Description
id	String	Audit identifier. Primary key.
store_id	String	Store affected by the action. Foreign key.
actor_user_id	String	User responsible; null for system actions. Foreign key. Optional field.
entity_type	String	Case, policy, membership, or related entity.
entity_id	UUID	Affected entity identifier.
action	String	Action or transition.
change_summary	JSON	Necessary before and after information.
occurred_at	Date/Time	Action time.

The database shall enforce unique store-domain associations, one active membership per store and user, and unique provider identifiers within their relevant store or channel. Order identifiers shall be unique per store rather than globally. Risk scores shall satisfy $0 \leq score \leq 1$. Staff assignments shall reference an active membership in the same store. Each store shall have at most one active policy version, activated only after successful indexing. Deletes shall not silently remove evidence needed by an existing complaint; retention and anonymization will use controlled operations.

4.10 Risk Analysis

Table 4.30 identifies technical and business risks with practical mitigations. Likelihood and impact are planning judgments rather than observed incident statistics.

Table 4.30: Technical and business risks and mitigations

Risk	Likelihood	Impact	Mitigation
Historical dataset mismatch	High	Risk scores may transfer poorly to current stores.	Inspect distributions, document the training population, evaluate on held-out outcomes, and avoid universal performance claims.

Risk	**Likelihood**	**Impact**	**Mitigation**
Missing or leaked customer history	Medium	Misleading scores or inflated evaluation results.	Build features in chronological order, use only prior records, represent missing history, and inspect prediction-time availability.
Limited bilingual labels	High	Poor category or urgency performance, especially on rare cases.	Define annotation rules, review disagreements, inspect per-class and language-group results, and retain human review.
Unsupported generated replies	Medium	Incorrect policy commitments or irrelevant advice.	Restrict retrieval by store and version, retain evidence, test conflicting policies, and route unsupported answers to staff.
Provider access or delivery failure	Medium	Incomplete synchronization or missed customer updates.	Verify permissions early, persist work, reconcile records, monitor failures, and retry eligible transient errors.
Inadequate courier fields	Medium	Courier performance cannot be calculated meaningfully.	Check field availability, define metric denominators, and display unavailable values rather than inferred performance.
Cross-store data exposure	Medium	Disclosure of customer or seller information.	Enforce server-side ownership checks, filter vector retrieval, protect credentials, and test unauthorized references.
Model latency and service cost	Medium	Slow replies or unaffordable operation.	Run inference asynchronously, reuse loaded models, bound generation requests, and measure cost and latency during evaluation.
Low seller adoption	Medium	A technically working tool may not improve the seller's process.	Validate the complaint view and risk-score presentation through task-based seller or agent feedback.

Risk	Likelihood	Impact	Mitigation
Integration scope growth	High	Work may exceed the two-semester schedule.	Maintain the agreed eight deliverables, complete one end-to-end workflow early, and defer new channels or transaction actions.

4.11 Conclusion

This chapter defined the functions, roles, quality targets, and use cases of Resolve AI, together with proposed resources, interface screenshots, and database records. The specifications maintain a clear distinction between return/refund-risk scoring and complaint assistance: predictions support review, while permitted replies and staff actions handle customer communication. The data model preserves order context, model evidence, policy versions, and case history so that the high-level and low-level design can implement these requirements consistently.

Chapter 5 Proposed Approach and Methodology

Resolve AI will be developed through an iterative approach that connects order-risk prediction with bilingual complaint assistance. This chapter outlines the preparation of data, development of predictive components, policy-grounded response generation, and integration and evaluation of the complete workflow.

5.1 Development Approach

The three-member team will complete the project over two semesters through short development iterations. Requirements and use cases will guide implementation, and each iteration will produce a reviewable component. Initial work will establish store connections, persistent records, and the staff interface. Subsequent iterations will integrate models, policy retrieval, routing, analytics, and WhatsApp communication. An early end-to-end workflow will help identify integration problems before all components are completed.

5.2 Data Preparation and Model Development

Pakistan's Largest E-Commerce Dataset will provide historical Pakistani transactions for return/refund-risk modelling. [8] Preparation will include examining available fields, defining the prediction target, removing duplicate records, handling missing values, and encoding categorical features. Available customer history will be computed using only records preceding the prediction time. Data will be separated into training, validation, and test sets, with chronological separation where suitable, to reduce leakage from future outcomes.

XGBoost and LightGBM will be considered for estimating the probability of a return or refund. Validation results will guide model selection and risk-tier thresholds; the held-out test set will remain separate from model tuning. The selected model will produce a score between zero and one, accompanied by its version and scoring time. Scores will support staff review.

For complaint analysis, English, Roman Urdu, and Urdu-English examples will be labelled by category and urgency using consistent annotation guidelines. Preparation will preserve meaningful language patterns while removing unnecessary personal information. A multilingual transformer, such as XLM-RoBERTa or mBERT, will be fine-tuned for classification, and a pretrained sentiment model will provide an additional signal. Performance will be examined separately for the supported language groups.

5.3 Policy-Grounded Complaint Resolution

Seller policies will be divided into passages and converted into sentence embeddings for storage in Qdrant. Each passage will retain its store and policy-version identifiers. When a complaint arrives through WhatsApp, the system will associate it with the relevant case, verify available order context, and obtain complaint analysis. Retrieval will select passages from the same store's active policy, and an LLM will use those passages and the case context to prepare a personalized response suggestion.

The routing component will consider urgency, confidence, sentiment, and policy evidence. Eligible routine cases may receive permitted automatic replies. Uncertain cases, missing evidence, and high-urgency complaints will require human review or escalation. Staff will be able to edit suggestions, take over conversations, and record case outcomes. The response and its supporting evidence will remain available in the case history.

5.4 Integration and Evaluation

Shopify events will be validated, stored, and processed asynchronously to update orders and request risk scoring. WhatsApp messages and delivery updates will be linked to the relevant conversations. The dashboard will summarize available order outcomes, risk tiers, complaint categories, urgency, and courier fields.

Evaluation will combine model assessment with functional testing. Risk assessment will use precision, recall, F1-score, and ROC-AUC; complaint classification will use macro F1-score and confusion matrices. Policy-based replies will be reviewed for relevance, grounding, and consistency with seller guidance. Integration checks will cover duplicate events, failed tasks, store isolation, staff actions, and message delivery states. Response times and task-based interface feedback will help assess usability. These are planned evaluation activities; measured results will be reported after testing.

5.5 Conclusion

The methodology combines iterative software development with local order data, bilingual complaint analysis, and policy-based assistance. It provides a practical route from the requirements to an integrated system, whose component design is presented in Chapter 6.

Chapter 6 High-Level and Low-Level Design

This chapter describes how Resolve AI will organize its components and process order data and customer complaints. The design follows the requirements in Chapter 4 and combines a web interface, background processing, predictive models, and policy-grounded complaint assistance.

6.1 System Overview

Resolve AI consists of eight connected modules: Shopify data ingestion, return/refund-risk prediction, bilingual complaint classification, sentiment analysis, policy-grounded response generation, case routing, analytics, and WhatsApp communication. Sellers and staff use the React interface, while customers communicate through WhatsApp. The backend connects these modules and maintains the order, policy, and conversation context required for each case.

6.2 Design Considerations

6.2.1 Assumptions and Dependencies

The design assumes an authorized Shopify connection, a configured WhatsApp business number, and seller-provided policies. Model development requires suitable order outcomes and labelled English and Roman Urdu complaint examples. External integrations depend on available API permissions and network connectivity.

6.2.2 General Constraints

A three-member team will develop the project over two semesters. Computing resources, model inference time, and external API limits influence the design. Customer information must remain within the correct store, and risk predictions provide review scores without executing return or refund transactions.

6.2.3 Goals and Guidelines

The design prioritizes clear interfaces, reusable components, consistent case history, and recoverable processing. Low-confidence analysis, missing policy evidence, and high-urgency complaints require human attention. Model outputs and staff corrections are stored separately to preserve traceability.

6.2.4 Development Methods

Development will proceed in short iterations with planning, implementation, review, and testing. The team will first connect a complete order-and-complaint workflow, then improve model quality and interface behaviour. Git will maintain source history, and tasks will be reviewed against the requirements in Chapter 4.

6.3 System Architecture

A modular backend is proposed to keep deployment manageable while separating integration, inference, and case-management responsibilities. The React interface communicates with FastAPI over HTTPS. Verified webhook events are persisted before being queued for background processing. PostgreSQL stores business records, while Qdrant stores policy embeddings. Figure 6.1 shows these connections.

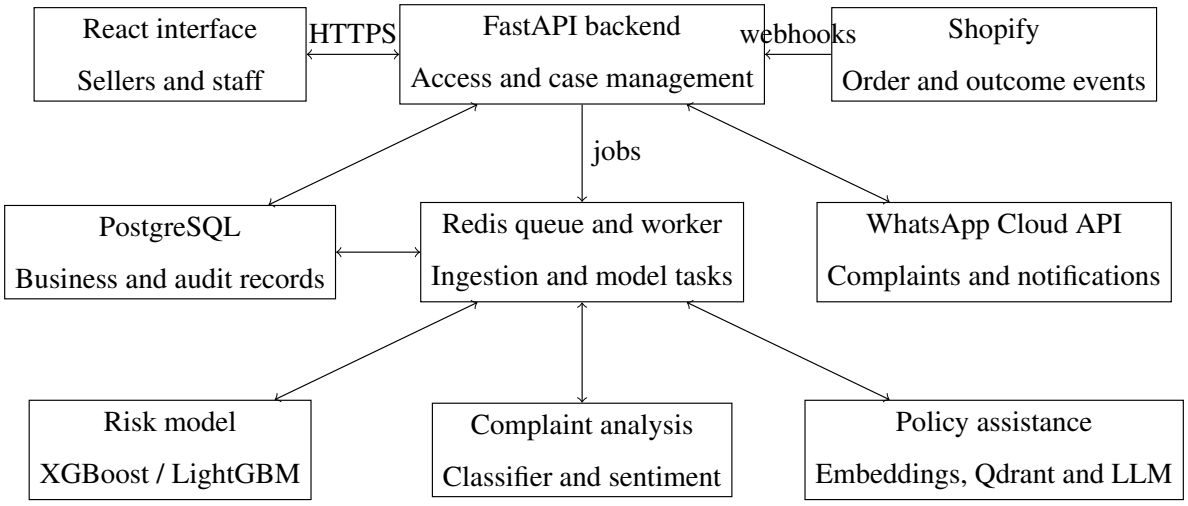


Figure 6.1: Proposed Resolve AI architecture showing staff access, external integrations, persistent storage, and background model processing

The ingestion component validates and normalizes provider events. The risk component produces a score and model-version record for supported orders. Complaint analysis produces category, urgency, sentiment, and confidence values. Policy assistance retrieves relevant passages before drafting a reply. Routing then selects permitted automatic handling, human review, or escalation. The dashboard summarizes stored records, and the messaging component records outgoing replies and provider delivery updates.

6.4 Architectural Strategies

React with TypeScript provides reusable interface components, and Python with FastAPI supports application logic and model integration. PostgreSQL maintains relational consistency; Qdrant supports store-filtered policy retrieval. Background workers handle slower inference and indexing tasks so webhook receipt and staff navigation remain responsive.

Provider adapters separate Shopify, WhatsApp, and LLM-specific calls from business logic. Duplicate event identifiers are checked before processing, failed jobs receive bounded retries, and unrecoverable failures remain visible to staff. Secrets are held on the server, access is checked against store membership, and containerized services provide a consistent deployment configuration.

6.5 Domain Model/Class Diagram

Figure 6.2 presents the application service classes, their operations, and their dependencies. ShopifyService queues order scoring through RiskPredictionService. WhatsAppService receives complaints, queues analysis, and invokes routing after analysis. RoutingService uses ResolutionService, which retrieves policy information through PolicyService and queues approved replies for WhatsAppService. AuthService, SupportService, and AnalyticsService provide authentication, staff intervention, and reporting responsibilities. Each class lists the entities it uses. Dashed arrows represent service dependencies; the persistent entities and their database relationships are described by the ER diagram and data dictionary in Chapter 4.

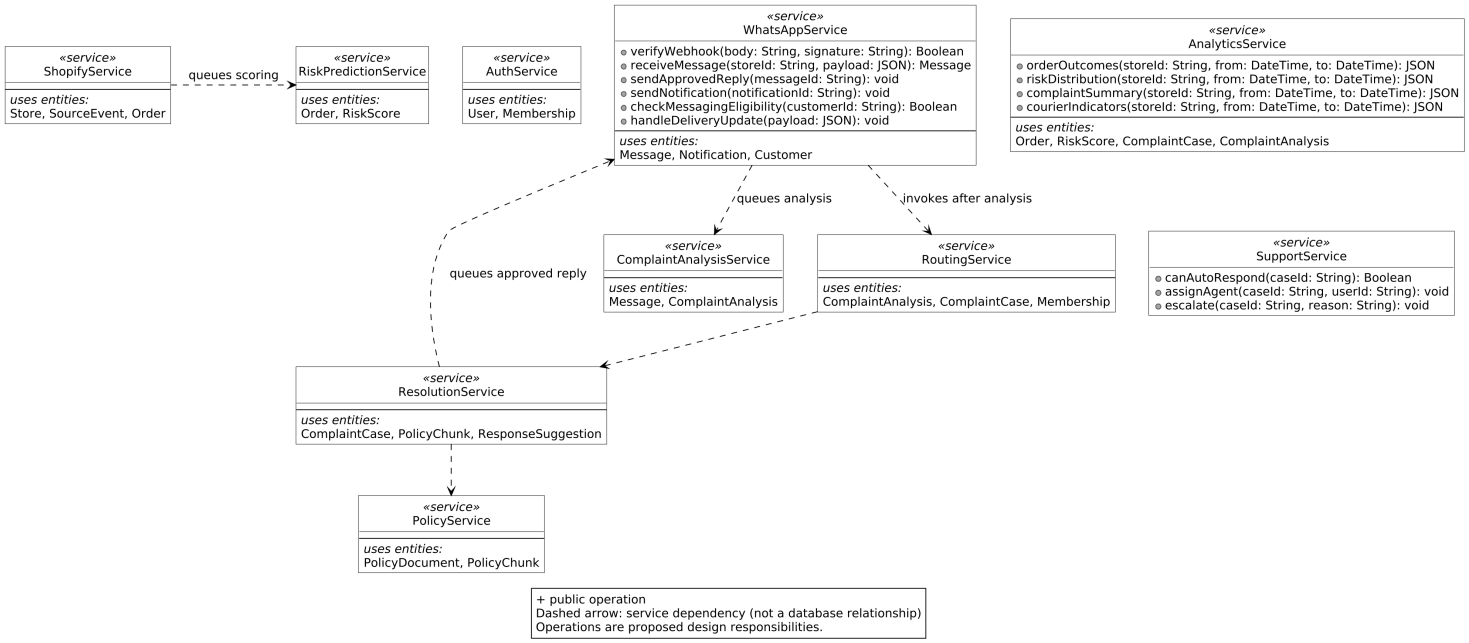


Figure 6.2: Resolve AI service class diagram showing application responsibilities, entity usage, and service dependencies

6.6 Sequence Diagram

Figure 6.3 shows the central complaint-handling workflow. A WhatsApp complaint is recorded and analysed for category, urgency, and sentiment. The backend uses linked order information and an existing return/refund-risk score when available, then requests a response grounded in the active seller policy. Routing selects a permitted automatic reply or staff review according to confidence, urgency, policy evidence, and human takeover. Delivery updates and case actions are recorded. Complaints can proceed without an order link or risk score; this workflow does not execute financial return or refund actions.

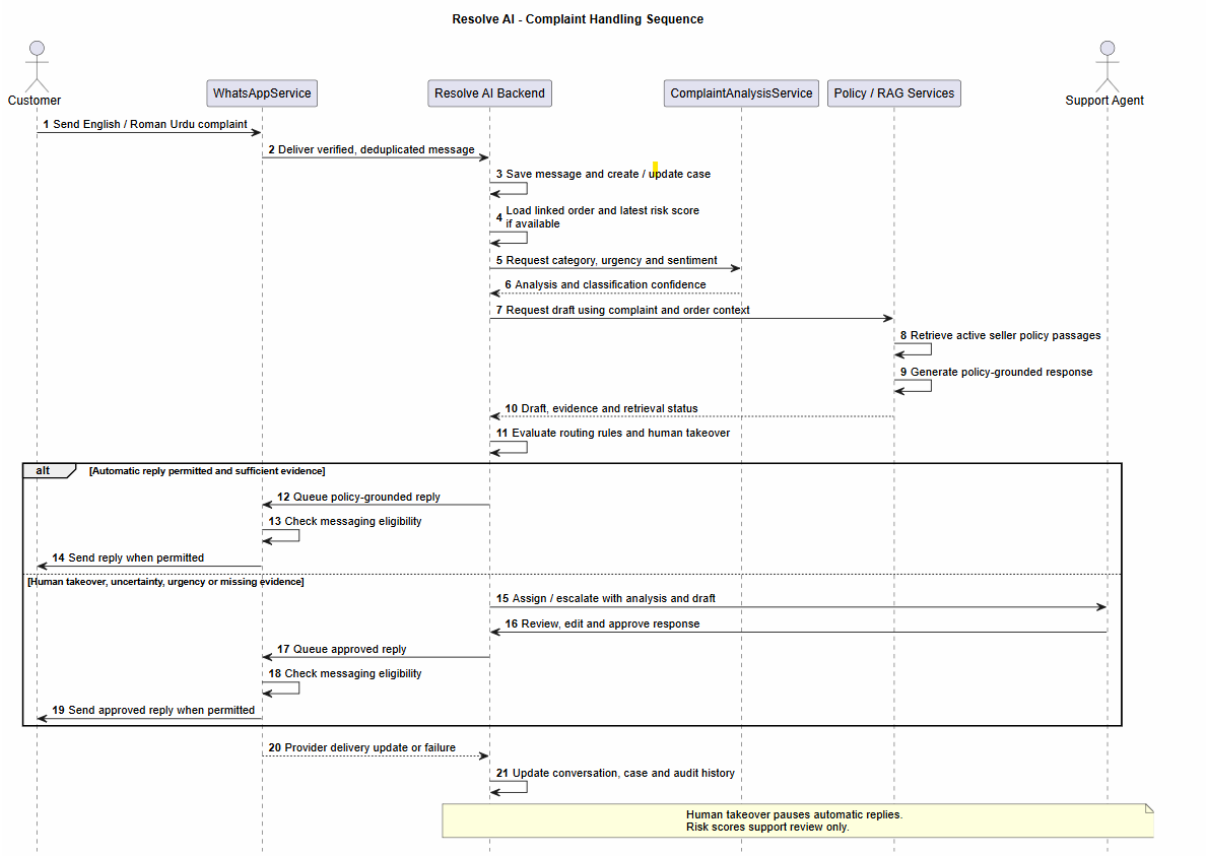


Figure 6.3: Complaint-handling sequence with policy-grounded response generation, conditional routing, staff review, and delivery recording

6.7 Policies and Tactics

Code will use consistent naming, typed API contracts, and separate modules for integrations, models, retrieval, and cases. Each prediction records its model version and uses only features available at scoring time. Policy retrieval filters by store and active version, and response evidence is retained with the draft.

Testing will cover duplicate webhooks, store-access isolation, routing conditions, unavailable scores, policy-version changes, and message failures. Model evaluation will use held-out data and report results separately for English and Roman Urdu complaints. Changes will be tracked against requirement identifiers, with migrations and configuration documented for maintenance.

6.8 Conclusion

The proposed design connects order-risk intelligence with bilingual complaint assistance through a modular backend and a consistent staff interface. Queued processing, policy evidence, store-scoped access, and human review provide the basis for implementation and subsequent evaluation.

Bibliography

[1] T. Chen and C. Guestrin, "XGBoost: A Scalable Tree Boosting System," in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016, pp. 785–794. doi: 10.1145/2939672.2939785. [Online]. Available: https://arxiv.org/abs/1603.02754.

[2] G. Ke et al., "LightGBM: A Highly Efficient Gradient Boosting Decision Tree," in *Advances in Neural Information Processing Systems*, vol. 30, 2017. [Online]. Available: https://papers.nips.cc/paper_files/paper/2017/hash/6449f44a102fde848669bdd9eb6b76fa-Abstract.html.

[3] T. Pires, E. Schlinger, and D. Garrette, "How Multilingual is Multilingual BERT?," in *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, 2019, pp. 4996–5001. doi: 10.18653/v1/P19-1493. [Online]. Available: https://aclanthology.org/P19-1493/.

[4] A. Conneau et al., "Unsupervised Cross-lingual Representation Learning at Scale," in *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, 2020, pp. 8440–8451. doi: 10.18653/v1/2020.acl-main.747. [Online]. Available: https://aclanthology.org/2020.acl-main.747/.

[5] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, 2019, pp. 3982–3992. doi: 10.18653/v1/D19-1410. [Online]. Available: https://aclanthology.org/D19-1410/.

[6] P. Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in *Advances in Neural Information Processing Systems*, vol. 33, 2020. [Online]. Available: https://proceedings.neurips.cc/paper/2020/hash/

6b493230205f780e1bc26945df7481e5-Abstract.html.

[7] F. Barbieri, L. Espinosa Anke, and J. Camacho-Collados, "XLM-T: Multilingual Language Models in Twitter for Sentiment Analysis and Beyond," in *Proceedings of the Thirteenth Language Resources and Evaluation Conference*, 2022, pp. 258–266. [Online]. Available: https://aclanthology.org/2022.lrec-1.27/.

[8] Z. Usmani, "Pakistan's Largest E-Commerce Dataset." Accessed: Oct. 5, 2026. [Online]. Available: https://www.kaggle.com/datasets/zusmani/pakistans-largest-ecommerce-dataset.

[9] Qdrant, "Filtering." Accessed: Oct. 5, 2026. [Online]. Available: https://qdrant.tech/documentation/search/filtering/.

[10] Loop, "Return Prediction Dashboard." Accessed: Oct. 5, 2026. [Online]. Available: https://help.loopreturns.com/en/articles/8795265.

[11] Zendesk, "Languages supported by AI agents." Accessed: Oct. 5, 2026. [Online]. Available: https://support.zendesk.com/hc/en-us/articles/8357749663130-Languages-supported-by-AI-agents.

[12] Zendesk, "About auto assist." Accessed: Oct. 5, 2026. [Online]. Available: https://support.zendesk.com/hc/en-us/articles/9945148867866-About-auto-assist.

[13] Gorgias, "AI Agent explained." Accessed: Oct. 5, 2026. [Online]. Available: https://docs.gorgias.com/en-US/ai-agent-explained-497772.

[14] Gorgias, "Launch guide for Gorgias admins." Accessed: Oct. 5, 2026. [Online]. Available: https://helpcenter.gorgias.com/en-US/launch-guide-for-gorgias-admins-7524979.

[15] Intercom, "Set up Fin for Ecommerce on a Shopify store." Accessed: Oct. 5, 2026. [Online]. Available: https://www.intercom.com/help/en/articles/14420740-set-up-fin-for-ecommerce-on-a-shopify-store.

[16] PKAILabs, "WhatsApp AI Assistant for Pakistani Businesses." Accessed: Oct. 5, 2026. [Online]. Available: https://www.pkailabs.io/whatsapp-ai.

[17] ShuttlePro, "ShuttlePro Intelligence." Accessed: Oct. 5, 2026. [Online]. Available: https://shuttlepro.io/shuttlepro-intelligence/.

[18] Shopify, “About webhooks.” Accessed: Oct. 5, 2026. [Online]. Available: https://shopify.dev/docs/apps/build/webhooks.

[19] FastAPI, “FastAPI.” Accessed: Oct. 5, 2026. [Online]. Available: https://fastapi.tiangolo.com/.